HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG

CÔNG NGHỆ THÔNG TIN 1

BÁO CÁO MÔN IOT VÀ ỨNG DỤNG

Chủ đề: Hệ thống IOT
giám sát môi trường phòng và điều khiển thiết bị

Bài báo cáo 1: SRS

Giảng viên hướng dẫn	**: TS. Nguyễn Quốc Uy**
Mã lớp	**: INT14149-11**
Họ và tên	**: Đặng Xuân Quang**
Mã sinh viên	**: B23DCCN686**

Hà nội – 2026

Mục lục

Chương 1: Giới thiệu đề tài 5
1.1. Đặt vấn đề 5
1.2. Lý do chọn đề tài 5
1.3. Mục tiêu đề tài 5
Chương 2: Thiết kế tổng quan 6
2.1. Kiến trúc hệ thống tổng thể 6
2.2. Lựa chọn thiết bị phần cứng (Hardware) 7
2.3. Lựa chọn công nghệ phần mềm (Software) 7
2.4. Các luồng hoạt động chính của hệ thống 7
2.4.1. Luồng thu thập dữ liệu 7
2.4.2. Luồng điều khiển thiết bị 8
2.5. Thiết kế giao diện người dùng 9
Chương 3: Thiết kế chi tiết 11
3.1. Đặc tả chi tiết các Usecase 11
3.1.1. UC Giám sát cảm biến theo thời gian 11
3.1.2. UC Giám sát biểu đồ cảm biến theo thời gian 14
3.1.3. UC Điều khiển thiết bị 17
3.1.4. UC Tra cứu lịch sử cảm biến 20
3.1.5. UC Tra cứu lịch sử thao tác thiết bị 23
3.2. Thiết kế cơ sở dữ liệu 28
1. Bảng User (Quản lý người dùng) 29
2. Bảng Devices (Danh sách thiết bị) 29
3. Bảng Sensors (Danh sách cảm biến) 29
4. Bảng Sensors_Data (Lịch sử dữ liệu cảm biến) 30
5. Bảng Actions_History (Nhật ký lịch sử Bật/Tắt thiết bị) 30
3.3. Tài liệu đặc tả API 31
3.3.1. Danh sách API tổng quan 31
3.3.2. Chi tiết đặc tả từng API 32
CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ THỰC NGHIỆM 40
4.1. Cấu trúc thư mục dự án 40

4.2. Kết quả triển khai phần cứng và MQTT 42
4.2.1. Triển khai phần cứng vi điều khiển 42
4.2.2. Giao thức truyền thông MQTT và quản lý phiên 42
4.3. Kết quả triển khai Backend và cơ sở dữ liệu 43
4.3.1. Cấu hình cơ sở dữ liệu MySQL 43
4.3.2. Tài liệu API và kiểm thử Endpoint 43
4.4. Kết quả xây dựng giao diện Frontend 44
4.5. Đánh giá thử nghiệm và kiểm thử hệ thống 46
4.6. Hướng dẫn chạy và triển khai hệ thống 46
4.6.1. Yêu cầu môi trường 46
4.6.2. Các bước triển khai chi tiết 46
CHƯƠNG 5: KẾT LUẬN 47
5.1. Kết luận và Kết quả đạt được 47
5.2. Bài học và Kinh nghiệm rút ra 47

Mục lục hình ảnh

Hình 2.1: Kiến trúc hệ thống tổng thể

Hình 2.5.1: Giao diện Dashboard

Hình 2.5.2: Giao diện Dữ liệu cảm biến

Hình 2.5.3: Giao diện Lịch sử bật tắt thiết bị

Hình 2.5.4: Giao diện Profile

Hình 3.1: Hình ảnh các Use case sủa hệ thống

Hình 3.1.1.2: Sequence Diagram giám sát cảm biến

Hình 3.1.1.3: Giao diện sử dụng API, Use case

Hình 3.1.2.2: Sequence Diagram giám sát biểu đồ cảm biến

Hình 3.1.2.3: Giao diện sử dụng API, Use case

Hình 3.1.3.2: Sequence Diagram Điều khiển thiết bị

Hình 3.1.3.3: Giao diện sử dụng API, Use case

Hình 3.1.4.2: Sequence Diagram Tra cứu lịch sử cảm biến

Hình 3.1.4.3: Giao diện sử dụng API, Use case

Hình 3.1.5.2: Sequence Diagram Tra cứu lịch sử thao tác thiết bị

Hình 3.1.5.3: Giao diện sử dụng API, Use case

Chương 1: Giới thiệu đề tài

1.1. Đặt vấn đề

Sự phát triển của Internet of Things (IoT) mở ra khả năng tự động hóa và giám sát môi trường sống theo thời gian thực. Trong các không gian như phòng ở, lớp học hoặc văn phòng, việc theo dõi các chỉ số môi trường (nhiệt độ, độ ẩm, ánh sáng) và quản lý thiết bị điện (đèn, quạt) thường gặp các hạn chế:

- Giám sát thủ công, thiếu tính liên tục và thiếu dữ liệu lịch sử để đánh giá chất lượng môi trường.
- Điều khiển thiết bị phụ thuộc vào công tắc cơ học tại chỗ, chưa hỗ trợ thao tác từ xa và chưa lưu vết lịch sử vận hành.
- Độ trễ trong hiển thị dữ liệu qua giao thức HTTP truyền thống còn cao, tiêu tốn tài nguyên mạng khi phải liên tục gửi truy vấn (polling).

1.2. Lý do chọn đề tài

- **Tính thực tiễn:** Xây dựng mô hình Smart Room hoàn chỉnh giúp tự động hóa việc thu thập chỉ số môi trường và điều khiển thiết bị điện qua Internet.
- **Tối ưu kiến trúc:** Ứng dụng giao thức MQTT kết hợp giảm tải băng thông mạng và phù hợp với phần cứng nhúng công suất thấp.
- **Làm chủ công nghệ:** Tích hợp đa nền tảng gồm phần cứng vi điều khiển (ESP8266/ESP32), giao thức IoT (MQTT), Backend (Node.js/Express), Cơ sở dữ liệu quan hệ (MySQL) và giao diện trực quan (ReactJS/TailwindCSS).

1.3. Mục tiêu đề tài

- Mục tiêu phần cứng: Thiết kế mạch phần cứng tích hợp vi điều khiển(ESP8266, NodeMCU), các cảm biến môi trường (DHT22, Quang trở LDR) và module Relay điều khiển thiết bị (Đèn, Đèn 2) an toàn, ổn định.
- **Mục tiêu phần mềm:** Xây dựng hệ thống Backend xử lý dữ liệu cảm biến, quản lý trạng thái thiết bị, cung cấp RESTful API.
- **Mục tiêu giao diện:** Phát triển Dashboard giám sát trực quan, hiển thị biểu đồ thời gian thực, bảng tra cứu lịch sử đo và nhật ký điều khiển.

Chương 2: Thiết kế tổng quan

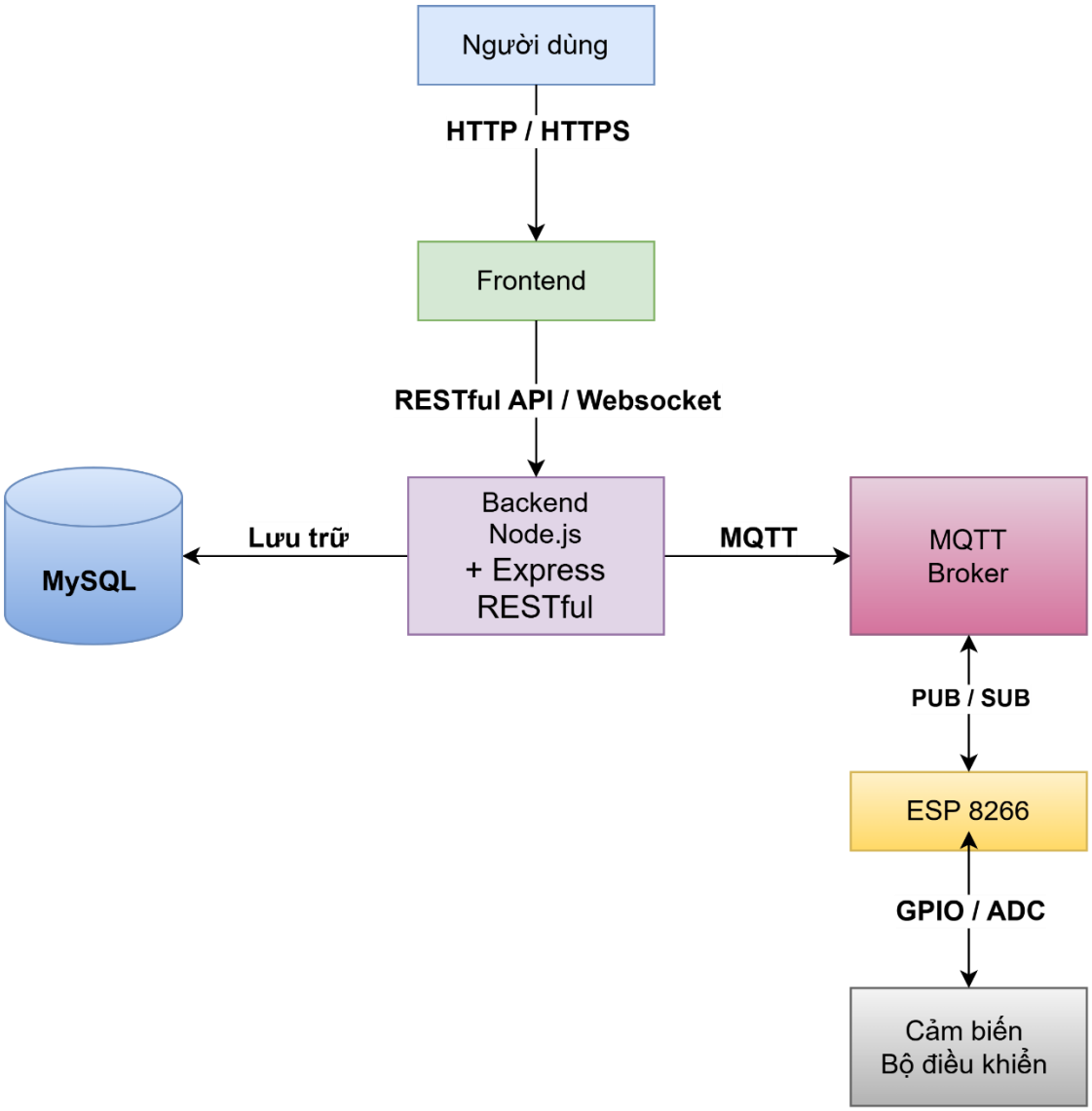
2.1. Kiến trúc hệ thống tổng thể

Hệ thống được thiết kế theo mô hình phân lớp đặc trưng của IoT, bao gồm 3 lớp chính:

- **Lớp thiết bị vật lý (Hardware):** Chịu trách nhiệm thu thập thông số môi trường trực tiếp từ không gian phòng và thực thi các lệnh điều khiển ngoại vi.

- **Lớp truyền thông (Transport Layer):** Đóng vai trò cầu nối trung chuyển giữa phần cứng và phần mềm, đảm bảo tốc độ cao vào tiêu thụ ít năng lượng

- **Lớp ứng dụng (Software):** Xử lý logic, lưu trữ dữ liệu tập trung và cung cấp giao diện tương tác cho người dùng cuối thông qua nền tảng Web



Hình 2.1: *Kiến trúc hệ thống tổng thể*

2.2. Lựa chọn thiết bị phần cứng (Hardware)

Lớp phần cứng được lựa chọn dựa trên tiêu chí tối ưu chi phí, khả năng xử lý ổn định và tích hợp sẵn chuẩn kết nối không dây:

- **Khối vi điều khiển trung tâm:** Sử dụng board phát triển **NodeMCU ESP8266**, tích hợp nhân xử lý 32-bit cùng module Wi-Fi chuẩn 802.11 b/g/n, đảm nhận vai trò thu thập tín hiệu ngoại vi và giao tiếp mạng.
- **Khối cảm biến thu thập môi trường (Sensor Inputs):**
    - *Cảm biến nhiệt độ & độ ẩm:* Module **DHT22** (hoặc DHT11) cho phép đo đạc liên tục các thông số nhiệt độ (°C) và độ ẩm không khí (%).
    - *Cảm biến ánh sáng:* Module quang trở **LDR** (hoặc cảm biến kỹ thuật số BH1750) chuyển đổi cường độ sáng môi trường (lux) thành tín hiệu điện áp/số.
- **Khối cơ cấu chấp hành (Actuator Outputs):** Sử dụng **Module Relay cách ly quang** kết hợp các đèn LED hiển thị nhằm mô phỏng và đóng/ngắt các thiết bị điện công suất (đèn chiếu sáng, quạt làm mát) một cách an toàn.

2.3. Lựa chọn công nghệ phần mềm (Software)

Kiến trúc phần mềm được phân tách độc lập giữa Client và Server đảm bảo khả năng mở rộng linh hoạt:

- **Máy chủ điều phối tin nhắn (MQTT Broker):** Triển khai dịch vụ **Eclipse Mosquitto** làm trạm trung chuyển điều phối các bản tin Publish/Subscribe với độ trễ thấp và kích thước gói tin tối ưu.
- **Tầng Backend & Lưu trữ dữ liệu:** Xây dựng trên nền tảng **Node.js/Express.js**, cung cấp các endpoint **RESTful API**. Hệ quản trị **Cơ sở dữ liệu quan hệ (MySQL)** chịu trách nhiệm lưu trữ có cấu trúc các chuỗi dữ liệu đo và lịch sử thao tác vận hành.
- **Tầng Giao diện người dùng (Frontend):** Xây dựng dưới dạng Single Page Application (SPA) với **ReactJS** và **TailwindCSS**, kết nối API để hiển thị Dashboard thời gian thực, vẽ đồ thị biến thiên và cung cấp bảng điều khiển trực quan.

2.4. Các luồng hoạt động chính của hệ thống

Hệ thống duy trì hai luồng xử lý độc lập nhằm đảm bảo tính liên tục của dữ liệu giám sát và tính toàn vẹn của lệnh điều khiển.

2.4.1. Luồng thu thập dữ liệu

Luồng này vận hành tự động theo chu kỳ khép kín để liên tục cập nhật trạng thái môi trường phòng:

- **Bước 1 (Đọc dữ liệu):** Vi điều khiển NodeMCU kích hoạt đọc định kỳ tín hiệu từ 3 cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng) theo chu kỳ thiết lập (2–3 giây/lần).
- **Bước 2 (Đóng gói Payload):** Dữ liệu thu thập được chuẩn hóa thành chuỗi JSON chứa định danh node và các thông số đo: {"deviceId": 1, "temp": 28.5, "humidity": 65.0, "light": 420.0, "timestamp": 1723647600}.
- **Bước 3 (Xuất bản tin nhắn):** NodeMCU đóng vai trò MQTT Publisher, phát gói tin lên topic sensor_data trên Mosquitto Broker.
- **Bước 4 (Xử lý & Lưu trữ):** Ứng dụng Backend (đóng vai trò MQTT Subscriber) tiếp nhận bản tin, bóc tách dữ liệu và thực thi lưu trữ vào bảng sensor_data trong Cơ sở dữ liệu.
- **Bước 5:** Backend đồng thời phát sự kiện qua kênh kết nối tới Frontend để giao diện cập nhật ngay lập tức các ô chỉ số và đồ thị mà không cần tải lại trang.

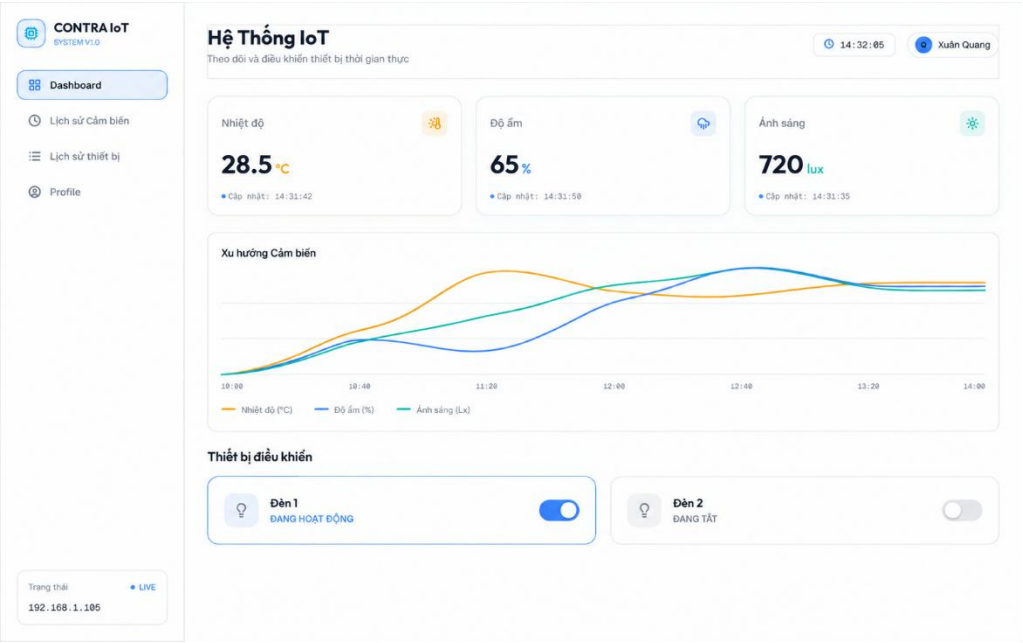
2.4.2. Luồng điều khiển thiết bị

Luồng điều khiển áp dụng cơ chế xác nhận phản hồi phần cứng (Hardware Acknowledgment - ACK) nhằm loại trừ triệt để tình trạng lệch trạng thái giữa giao diện và thực tế:

- **Bước 1 (Khởi tạo lệnh):** Người dùng gạt công tắc Bật/Tắt thiết bị trên Dashboard. Giao diện Frontend lập tức kích hoạt trạng thái tải (Loading State) và tạm khóa nút bấm để chống thao tác liên tục.
- **Bước 2 (Ghi nhận yêu cầu):** Frontend gửi HTTP POST tới API điều khiển của Backend. Backend ghi nhận một bản ghi thao tác mới vào bảng actions với trạng thái ban đầu là PENDING.
- **Bước 3 (Chuyển tiếp lệnh qua MQTT):** Backend đóng gói mã lệnh điều khiển kèm mã định danh yêu cầu (requestId) và Publish lên topic device_control.
- **Bước 4 (Thực thi phần cứng):** Vi điều khiển nhận lệnh từ Broker, giải mã payload và xuất mức logic tương ứng ra chân GPIO để kích hoạt Relay đóng/ngắt thiết bị vật lý.
- **Bước 5 (Phản hồi xác nhận):** Sau khi trạng thái chân GPIO thay đổi thành công, vi điều khiển Publish một gói tin phản hồi trạng thái (ACK) kèm status: "SUCCESS" lên topic device_response.
- **Bước 6 (Cập nhật và hoàn tất):** Backend tiếp nhận ACK, cập nhật trạng thái bảng actions thành SUCCESS, đồng bộ trạng thái thiết bị trong bảng devices thành ON (hoặc OFF), sau đó phản hồi kết quả về Frontend để mở khóa nút bấm và hiển thị trạng thái chính thức lên màn hình.

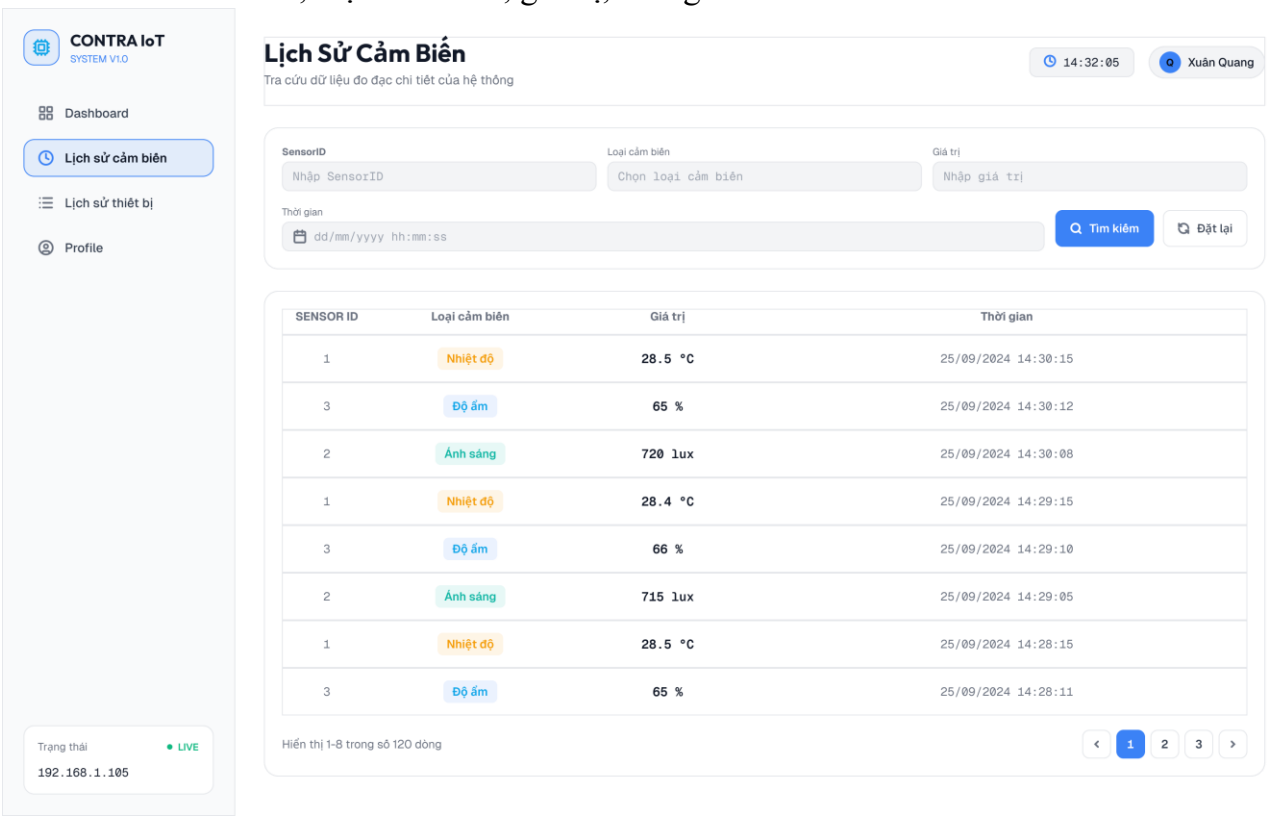
2.5. Thiết kế giao diện người dùng

Giao diện Dashboard: Hiển thị trực quan chỉ số của 3 cảm biến cùng cùng biểu đồ giao động và khu vực nút bấm điều khiển bật/tắt 2 thiết bị



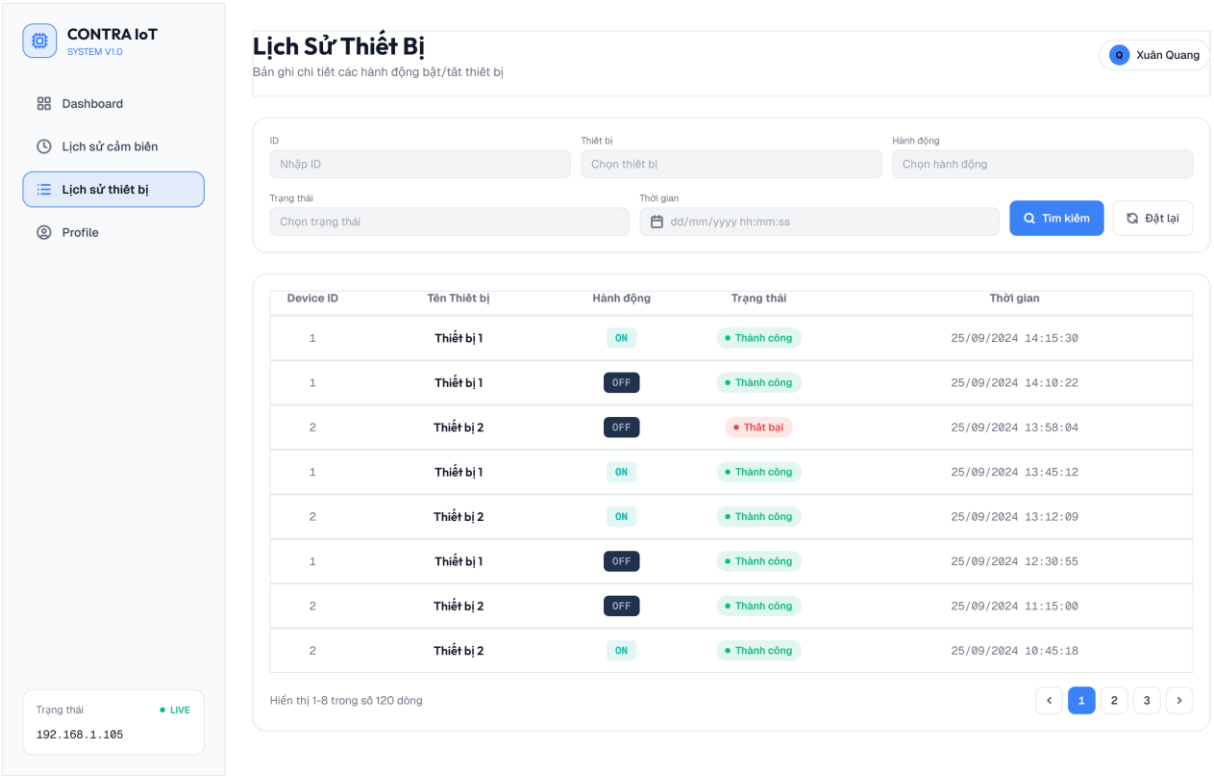
Hình 2.5.1: *Giao diện Dashboard*

Giao diện Dữ liệu cảm biến: Hiển thị bản ghi dữ liệu các cảm biến chi tiết và phần tìm kiếm chi tiết theo: ID, loại cảm biến, giá trị, thời gian.



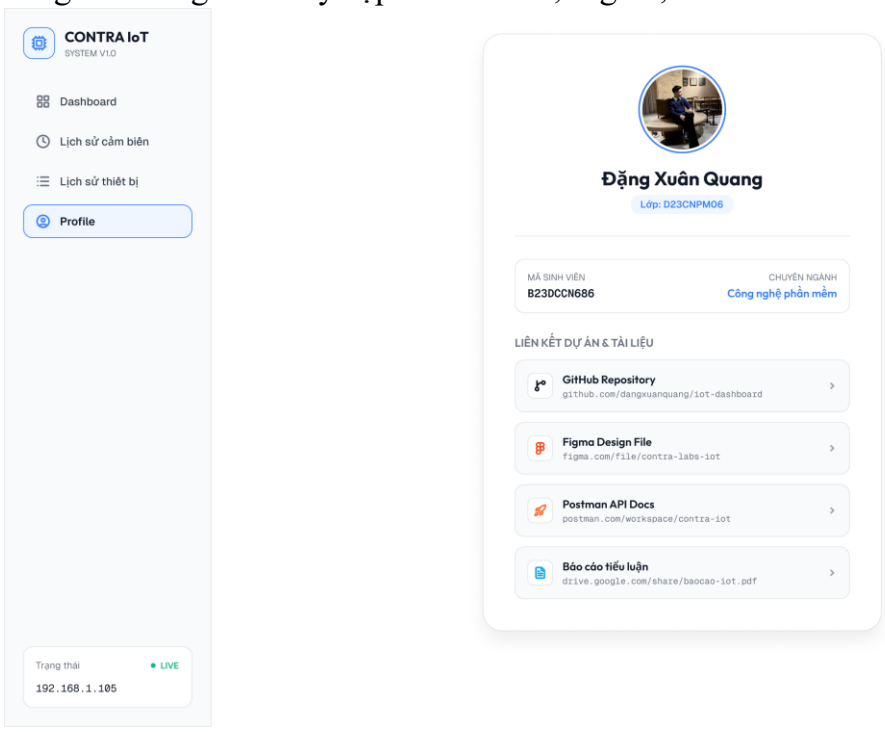
Hình 2.5.2*: Giao diện Dữ liệu cảm biến*

Giao diện Lịch sử bật tắt thiết bị: Hiển thị chi tiết lịch sử Bật/Tắt và trạng thái của thiết bị, cho phép tìm chi tiết theo: ID, tên thiết bị, hành động, trạng thái, thời gian.



Hình 2.5.3: *Giao diện Lịch sử bật tắt thiết bị*

Giao diện Profile: Giới thiệu thông tin người dùng avatar, họ và tên, mã sinh viên cùng cá đường link truy cập tới: Github, Figma, Postman API Docs, báo cáo pdf.

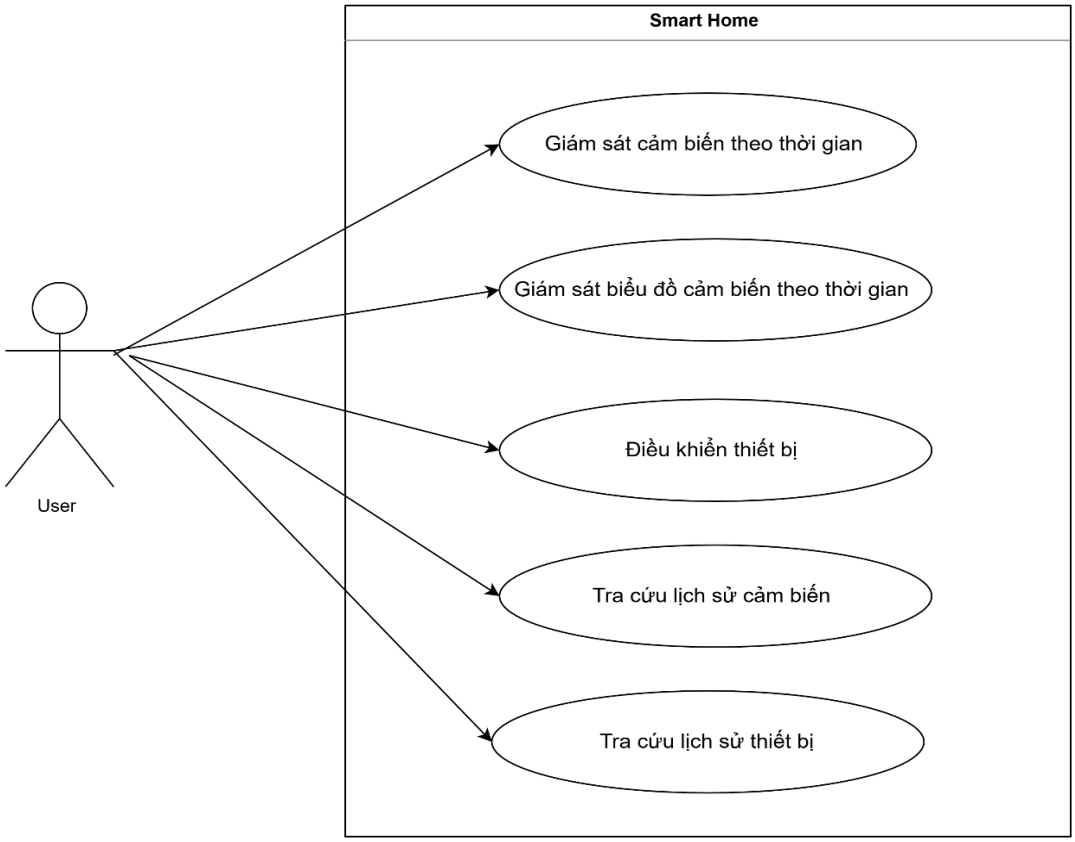


Hình 2.5.4: *Giao diện Profile*

Chương 3: Thiết kế chi tiết

3.1. Đặc tả chi tiết các Usecase

Các Use case bao gồm:



Hình 3.1: *Hình ảnh các Use case sủa hệ thống*

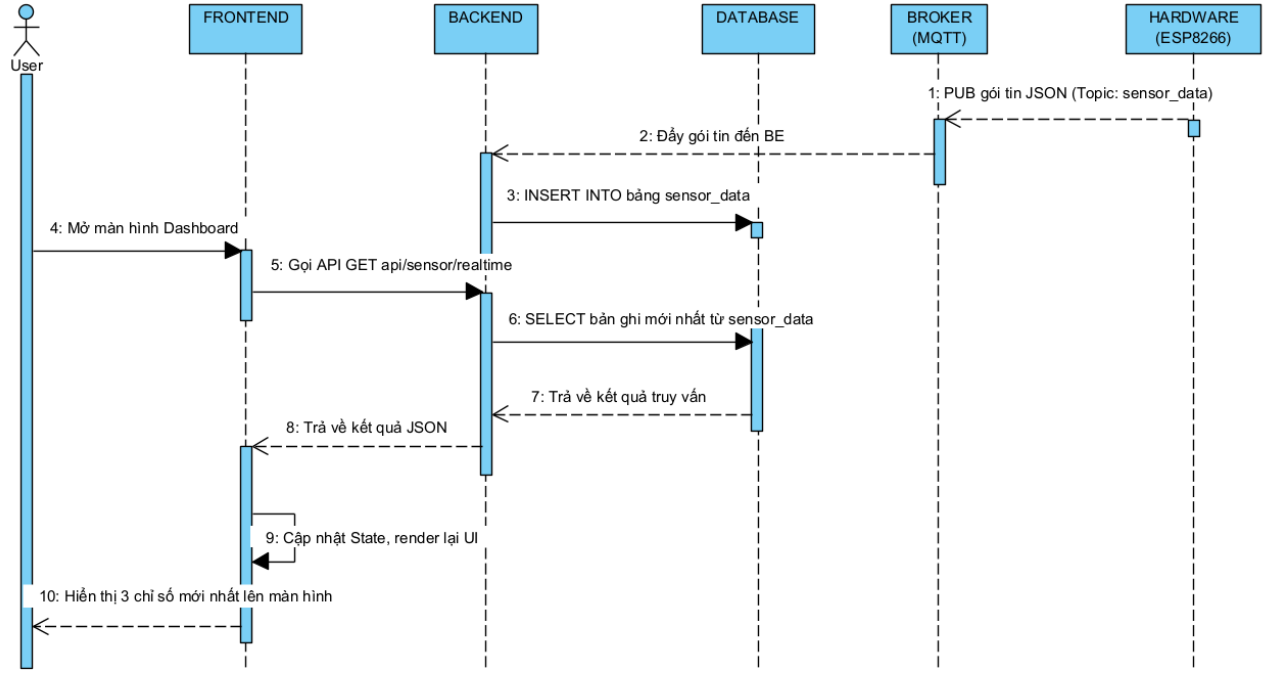
3.1.1. UC Giám sát cảm biến theo thời gian

3.1.1.1. Mô tả

Mã UC	UC – 01
Tên Use case	Giám sát cảm biến theo thời gian thực
Tác nhân	User
Mô tả	Người dùng khi mới vào Dashboard có thể xem được các chỉ số của cảm biến
Tiền điều kiện	Mạch vi điều khiển (ESP8266/NodeMCU) đã khởi động và kết nối thành công với MQTT Broker. Hệ thống Backend đã kết nối và dăng kí nhận dữ liệu thành công topic sensor_data
Hậu điều kiện	Người dùng xem được các chỉ số của Các cảm biến

Luồng sự kiện chính	1. Người dùng truy cập vào hệ thống IoT quản lý trên trình duyệt Web 2. Giao diện FE gửi request lấy dữ liệu cảm biến mới nhất từ BE để hiển thị 3 thẻ Widget (Nhiệt độ, Độ ẩm, Ánh sáng) 3. FE khởi tạo kết nối tới BE để chuẩn bị nhận dữ liệu luồng 4. Phần cứng định kỳ đo và đẩy dữ liệu lên MQTT Broker, BE nhận tin, lưu vào Cơ sở dữ liệu và chuyển tiếp qua xuống FE 5. Người dùng quan sát thấy các chỉ số trên màn hình tự động thay đổi theo chu kỳ
Luồng ngoại lệ	1. **Mất kết nối thiết bị phần cứng:** Nếu quá 15 giây Backend không nhận được dữ liệu mới từ MQTT Broker Backend đẩy sự kiện NODE_OFFLINE lên Frontend, FE chuyển trạng thái hiển thị cảnh báo 2. **Đứt kết nối:** FE hiển thị mất kết nối và tự động kết nối lại

3.1.1.2. Sơ đồ tuần tự



Hình 3.1.1.2: *Sequence Diagram giám sát cảm biến*

Mô tả chi tiết luồng:

1. **Vi điều khiển** (HW) đọc định kỳ 3 cảm biến (nhiệt độ, độ ẩm, ánh sáng), đóng gói chuỗi JSON và PUB lên MQTT Broker qua topic sensor_data.
2. **MQTT Broker** chuyển tiếp gói tin về Backend do Backend đã SUB topic này từ trước.
3. **Backend** bóc tách chuỗi JSON và thực thi lệnh SQL INSERT để lưu bản ghi mới vào bảng sensor_data trong Database.
4. Người dùng thao tác mở màn hình Dashboard trên trình duyệt web.
5. **Frontend** tự động gọi API GET /api/sensor/realtime định kì 2s/lần gửi tới Backend để lấy dữ liệu cảm biến gần nhất khởi tạo giao diện.
6. **Backend** nhận yêu cầu, gửi truy vấn SELECT lấy các bản ghi có thời gian mới nhất trong bảng sensor_data từ Cơ sở dữ liệu.
7. **Cơ sở dữ liệu** trả lại kết quả bản ghi cảm biến mới nhất cho Backend;
8. **Backend** chuẩn hóa thành chuỗi JSON và phản hồi lại cho Frontend thông qua API
9. **Frontend** nhận dữ liệu JSON, tiến hành thay đổi biến trạng thái (state) và render lại UI
10. Giao diện hoàn thành việc thay đổi. người dùng nhìn thấy 3 chỉ số mới nhất của cảm biến trên giao diện

3.1.1.3. API lấy dữ liệu cảm biến gần nhất cho Dashboard

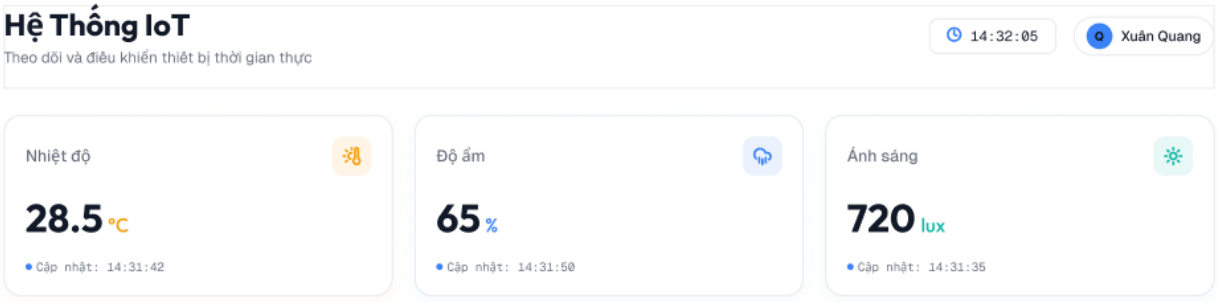
Mô tả: Lấy giá trị cảm biến cập nhật gần nhất để hiển thị lên 3 ô chỉ số và vẽ biểu đồ thời gian thực trên màn hình Dashboard.

- **Method:** GET
- **Endpoint:** /api/v1/sensor/realtime
- **Headers:** Authorization: Bearer <token>
- **Response (JSON):**
  JSON

  ```
  {
    "status": "success",
    "data": {
      "temperature": 28.0,
      "humidity": 80.5,
      "light": 1124,
  ```

```
    "timestamp": "2026-08-25T10:05:02Z"
  }
}
```


Màn hình sử dụng API và Use case:



Hình 3.1.1.3: *Giao diện sử dụng API, Use case*

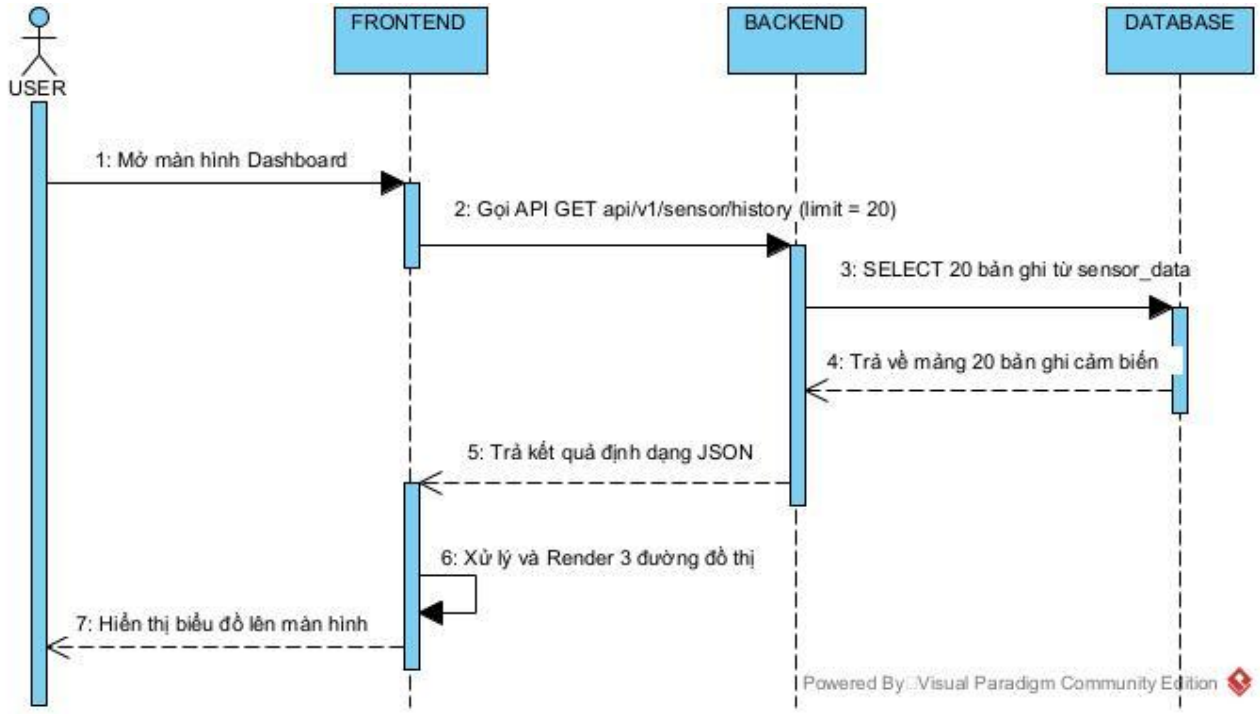
3.1.2. UC Giám sát biểu đồ cảm biến theo thời gian

3.1.2.1. Mô tả

Mã UC	UC – 02
Tên Use case	Giám sát biểu đồ cảm biến theo thời gian thực
Tác nhân	User
Mô tả	Người dùng khi mới vào Dashboard có thể xem được biểu đồ cảm biến theo thời gian thực
Tiền điều kiện	Hệ thống Backend (BE) đang hoạt động ổn định. Bảng sensor_data trong Cơ sở dữ liệu (DB) đã có sẵn các bản ghi dữ liệu cảm biến được phần cứng (HW) đẩy lên trước đó.
Hậu điều kiện	Người dùng xem được biểu đồ các chỉ số của Các cảm biến
Luồng sự kiện chính	1. Người dùng đang ở màn hình Dashboard sau khi đăng nhập thành công, muốn xem biểu đồ cảm biến trên giao diện 2. Giao diện (FE) gửi yêu cầu lấy mảng dữ liệu từ Backend để dựng khung biểu đồ ban đầu 3. Backend truy vấn Cơ sở dữ liệu và trả về mảng dữ liệu 4. FE render biểu đồ đường tương tác hiển thị sự giao động của các chỉ số

Luồng ngoại lệ	1. **Cơ sở dữ liệu trống:** Nếu chưa có bản ghi nào, Backend trả về mảng trống. FE bắt ngoại lệ thông báo “Chưa có dữ liệu cảm biến” 2. **Lỗi kết nối/Server Error:** Nếu Backend phản hồi mã lỗi 500, FE hiển thị thông báo "Không thể tải biểu đồ lúc này" và hiển thị nút "Thử lại".

3.1.2.2. Sơ đồ tuần tự



Hình 3.1.2.2: *Sequence Diagram giám sát biểu đồ cảm biến*

Mô tả chi tiết luồng:

1. Người dùng truy cập vào giao diện Dashboard và quan sát biểu đồ trên giao diện Dashboard
2. **Frontend** gọi API GET /api/v1/sensors/history?deviceId=1&limit=20 tới Backend để yêu cầu lấy mảng dữ liệu lịch sử đo lường gần nhất.
3. Backend tiếp nhận API, bóc tách tham số và thực thi lệnh SELECT kết hợp ORDER BY time DESC LIMIT 20 vào bảng sensor_data trong DB.
4. **DB** xử lý câu lệnh truy vấn và trả lại một mảng gồm 20 bản ghi cảm biến cho Backend.
5. **Backend** chuẩn hóa mảng bản ghi này thành định dạng mảng JSON và trả về cho Frontend.
6. **Frontend** tiếp nhận mảng JSON, xử lý dữ liệu thành các tọa độ và render 3 đường thẳng đại diện cho nhiệt độ, độ ẩm, ánh sáng.

7. Quá trình render hoàn tất, giao diện hiển thị biểu đồ hoàn chỉnh lên màn hình cho người dùng quan sát.

3.1.2.3. API lấy dữ liệu cho biểu đồ cảm biến thời gian thực

Mô tả: Lấy mảng dữ liệu đo lường gần nhất (ví dụ: 20 bản ghi) để vẽ biểu đồ đường mô phỏng sự biến thiên các chỉ số.

Method: GET

Endpoint: /api/v1/sensors/history

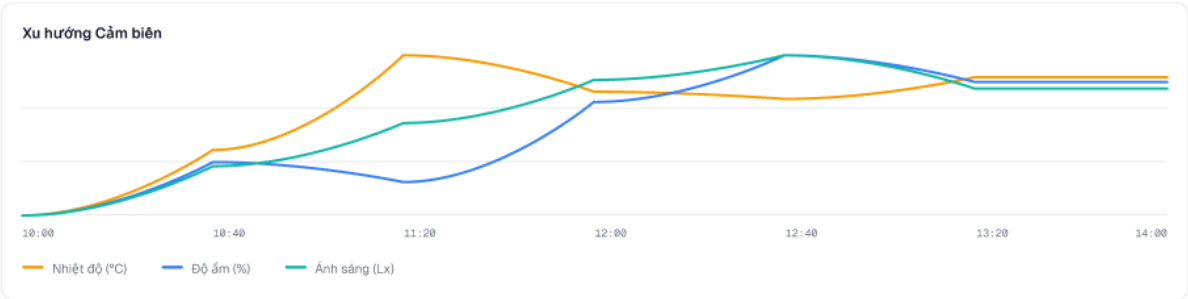
Headers: Authorization: Bearer <token>

Query Parameters: deviceId=1, limit=20

Response (JSON - 200 OK):


```
{
 "status": "success",
 "data": [
  {
   "temperature": 27.5,
   "humidity": 80.0,
   "light": 1010,
   "time": "2026-08-25T10:00:00Z"
  },
  {
   "temperature": 28.0,
   "humidity": 80.5,
   "light": 1124,
   "time": "2026-08-25T10:05:02Z"
  }
 ]
}
```


Màn hình sử dụng API và Use case:



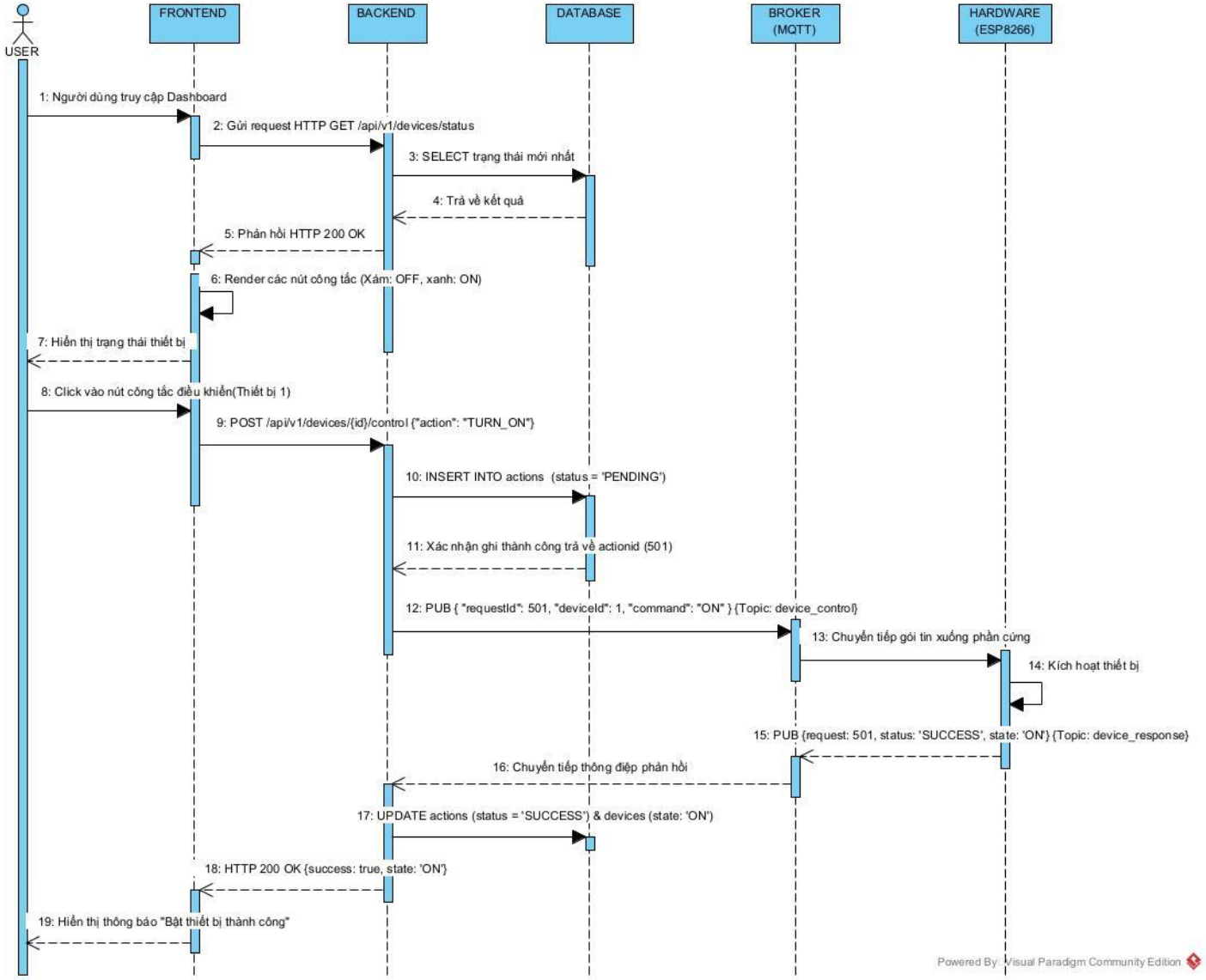
Hình 3.1.2.3: *Giao diện sử dụng API, Use case*

3.1.3. UC Điều khiển thiết bị

3.1.3.1. Mô tả

Mã UC	UC – 03
Tên Use case	Điều khiển thiết bị
Tác nhân	User
Mô tả	Cho phép người dùng ra lệnh bật tắt thiết bị tữa qua giao diện Web
Tiền điều kiện	Mạch vi điều khiển (ESP8266) đang trực tuyến, kết nối MQTT Broker và đã Subscribe topic device_control. Backend Server đang kết nối MQTT Broker và đã Subscribe topic device_response
Hậu điều kiện	Thiết bị hoạt động theo điều khiển của người dùng
Luồng sự kiện chính	1. Quản trị viên gạt nút công tắc Bật/Tắt của Đèn (hoặc Quạt) trên Dashboard. 2. Frontend chuyển nút bấm sang trạng thái loading và tạm thời vô hiệu hóa để tránh spam click. 3. Frontend gửi request điều khiển tới Backend; Backend tạo bản ghi actions với trạng thái PENDING và Publish lệnh xuống MQTT Broker. 4. Vi điều khiển nhận lệnh, xuất tín hiệu GPIO kích hoạt Relay và gửi phản hồi xác nhận SUCCESS lên Broker. 5. Backend nhận phản hồi, cập nhật CSDL sang SUCCESS và ON/OFF, trả HTTP 200 OK về cho FE. 6. FE tắt loading, chuyển nút gạt sang trạng thái mới và hiển thị thông báo thành công.
Luồng ngoại lệ	1. **Timeout – Phần cứng không phản hồi:** Nếu sau 3–5 giây vi điều khiển không phản hồi gói tin device_response, Backend cập nhật bản ghi actions thành FAILED và trả mã lỗi 504 Gateway Timeout 2. **Thiết bị không hợp lệ:** Backend phản hồi mã lỗi 400 hoặc 403. FE hoàn tác nút bấm và hiển thị cảnh báo lỗi tương ứng.

3.1.3.2. Sơ đồ tuần tự



Hình 3.1.3.2: *Sequence Diagram điều khiển thiết bị*

Mô tả chi tiết luồng:

1. Người dùng truy cập trang Dashboard
2. **Frontend** gửi request HTTP GET /api/v1/devices/status tới Backend Server để lấy trạng thái hiện tại của tất cả thiết bị.
3. **Backend** Thực hiện truy vấn Database để lấy trạng thái thiết bị hiện tại
4. **Database** trả về kết quả
5. Backend và phản hồi HTTP 200 OK { "data": [{ "id": 1, "name": "Đèn", "status": "OFF" }, { "id": 2, "name": "Đèn 2", "status": "ON" }] }

6. **Frontend** nhận dữ liệu và render trạng thái thực tế lên các nút công tắc (Nút màu xám thể hiện trạng thái OFF, nút màu xanh thể hiện trạng thái ON)
7. Hiển thị trạng thái thiết bị ra **Giao diện**
8. **Người dùng** click vào nút công tắc điều khiển của Thiết bị (Đèn1/2) trên giao diện Dashboard.
9. **Frontend** gọi API POST /api/v1/devices/{id}/control kèm Request Body { "action": "TURN_ON" } gửi tới Backend Server.
10. **Backend** thực thi lệnh SQL INSERT tạo một bản ghi thao tác mới vào bảng actions với trạng thái ban đầu là PENDING.
11. **Cơ sở dữ liệu** xác nhận ghi thành công và trả về actionId vừa tạo cho Backend.
12. **Backend** đóng gói mã lệnh điều khiển thành chuỗi JSON { "requestId": 501, "deviceId": 1, "command": "ON" } và PUB lên MQTT Broker qua topic device_control.
13. **MQTT Broker** nhận gói tin và chuyển tiếp trực tiếp xuống vi điều khiển phần cứng (NodeMCU/ESP8266).
14. **Vi điều khiển** giải mã gói tin JSON, nhận diện mã lệnh và xuất mức điện áp HIGH ra chân GPIO chỉ định để kích hoạt cuộn hút Relay đóng điện cho tải thực tế.
15. Sau khi Relay kích hoạt thành công, vi điều khiển đóng gói phản hồi xác nhận { "requestId": 501, "status": "SUCCESS", "state": "ON" } và PUB lên Broker qua topic device_response.
16. **MQTT Broker** chuyển tiếp thông điệp phản hồi từ phần cứng về cho Backend Server.
17. **Backend** tiếp nhận phản hồi, thực thi câu lệnh **SQL** UPDATE actions SET status = 'SUCCESS' WHERE id = requestId và UPDATE devices SET state= 'ON' WHERE id = deviceId vào Cơ sở dữ liệu.
18. **Backend** gửi phản hồi HTTP 200 OK { success: true, state: "ON" } về cho Frontend.
19. **Frontend** tiếp nhận kết quả đổi màu công tắc sang màu xanh (ON) và hiển thị thông báo "Bật thiết bị thành công" cho Quản trị viên.

3.1.3.3. API Điều khiển thiết bị

Mô tả: Gửi yêu cầu bật/tắt thiết bị ngoại vi qua REST API.

Method: POST

Endpoint: /api/v1/devices/{id}/control

Headers: Authorization: Bearer <token>, Content-Type: application/json

Request Body (JSON):

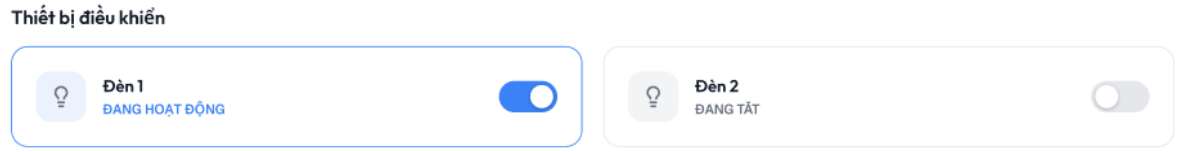
{

```
  "action": "TURN_ON"
}
```


Response (JSON - 200 OK):


```
{
  "status": "success",
  "message": "Điều khiển thiết bị thành công",
  "data": {
    "action_id": 501,
    "device_id": 1,
    "state": "ON",
    "updated_at": "2026-08-25T10:06:00Z"
  }
}
```


Màn hình sử dụng API và Use case:



Hình 3.1.3.3: *Giao diện sử dụng API, Use case*

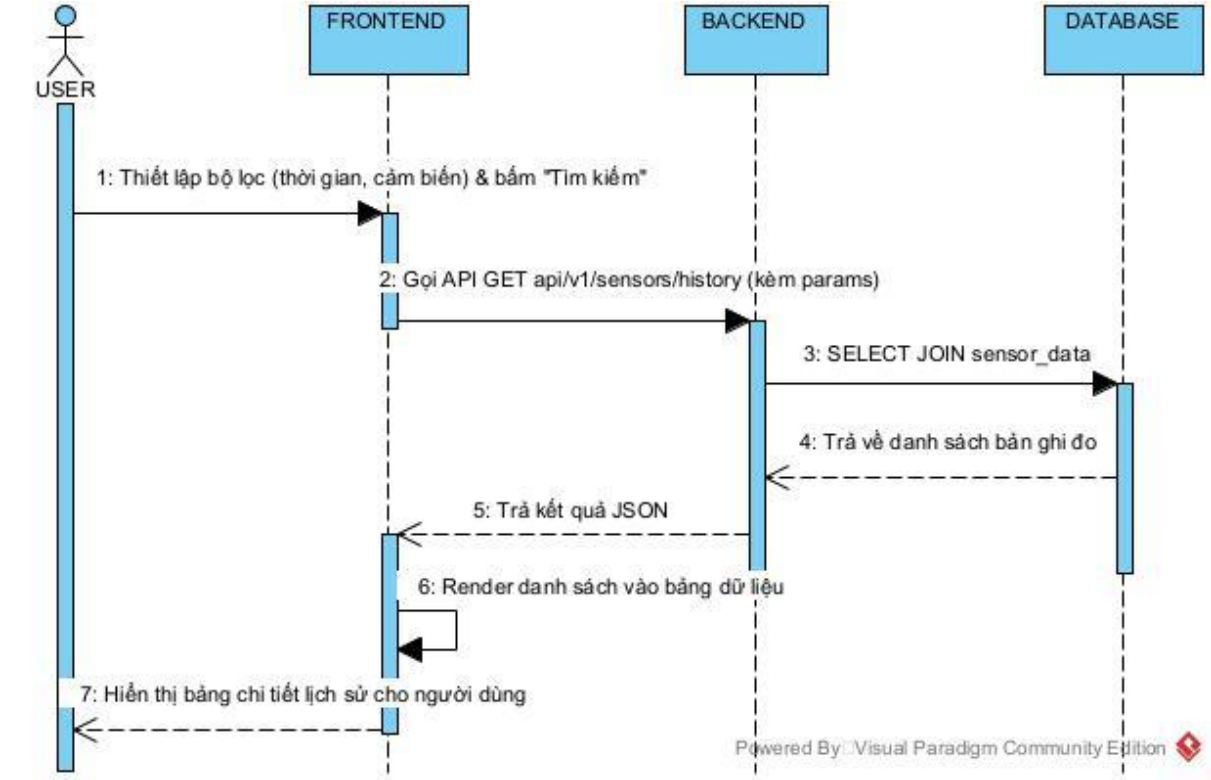
3.1.4. UC Tra cứu lịch sử cảm biến

3.1.4.1. Mô tả

Mã UC	UC – 04
Tên Use case	Tra cứu lịch sử cảm biến
Tác nhân	User
Mô tả	Tìm kiếm, lọc và xem lại các giá trị đo lường của cảm biến trong quá khứ dựa trên các mốc thời gian tùy biến
Tiền điều kiện	Hệ thống Backend đang hoạt động ổn định. Bảng sensor_data trong Cơ sở dữ liệu đã lưu trữ các bản ghi lịch sử đo.
Hậu điều kiện	Người dùng xem được dữ liệu các bản ghi của cảm biến
Luồng sự kiện chính	1. Quản trị viên truy cập màn hình "Lịch sử cảm biến". 2. Quản trị viên thiết lập bộ lọc: chọn loại cảm biến, thời gian, ... sau đó nhấn “Tìm kiếm” 3. Frontend gửi request tra cứu kèm các tham số lọc tới Backend.

	4. Backend truy vấn kết hợp bảng sensor_data và sensors, thực hiện phân trang và trả về danh sách dữ liệu kèm metadata phân trang. 5. Frontend kết xuất bảng dữ liệu chi tiết ra màn hình kèm thanh chuyển trang.
Luồng ngoại lệ	1. **Không tìm thấy dữ liệu:** Nếu không có bản ghi nào khớp với bộ lọc, Backend trả về mảng rỗng [] kèm total_records = 0. FE hiển thị bảng rỗng với thông báo "***Không có dữ liệu đo nào phù hợp với điều kiện tìm kiếm***". 2. **Lỗi kết nối:** FE hiển thị thông báo "Không thể tải dữ liệu cảm biến" và cung cấp nút "Tải lại"

3.1.4.2. Sơ đồ tuần tự



***Hình 3.1.4.2**: Sequence Diagram tra cứu lịch sử cảm biến*

Mô tả chi tiết luồng:

1. **Người dùng** truy cập vào menu "Lịch sử cảm biến" trên thanh điều hướng của hệ thống. Người dùng thiết lập bộ lọc thời gian và nhấn tìm kiếm
2. **Frontend** kiểm tra tính hợp lệ và gọi API api/v1/sensors/history?time gửi tới Backend

3. **Backend** bóc tách tham số lọc xử lý câu lệnh truy vấn SQL vào bảng sensor_data
4. **Cơ sở dữ liệu** xử lý truy vấn và trả lại danh sách các bản ghi dữ liệu đo kèm tổng số lượng bản ghi cho Backend.
5. **Backend** chuẩn hóa mảng dữ liệu này thành định dạng mảng JSON (bao gồm id, sensorID, name, value, time) và phản hồi lại cho Frontend.
6. **Frontend** tiếp nhận mảng JSON, thực thi các hàm xử lý dữ liệu và hiển thị giao diện.
7. **Giao diện** hoàn thành việc render, hiển thị bảng danh sách giá trị cảm biến chi tiết lên màn hình cho người dùng xem

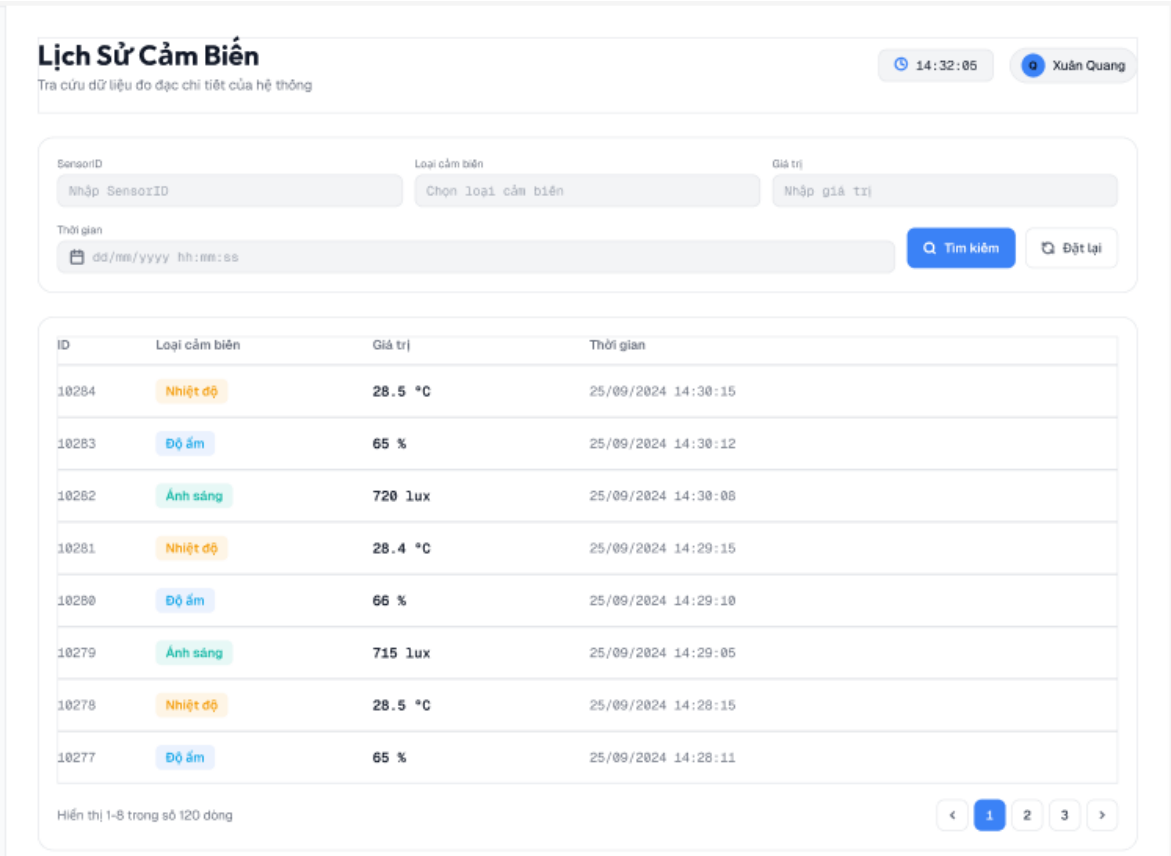
3.1.4.3. API Xem danh sách và tra cứu lịch sử cảm biến

- **Mô tả:** Lấy danh sách lịch sử các giá trị cảm biến, hỗ trợ phân trang và tìm kiếm theo mốc thời gian (time).
- **Method:** GET
- **Endpoint:** /api/v1/sensor/history
- **Headers:** Authorization: Bearer <token>
- **Query Parameters:** sensor_id=1, time=2026-08-25 10:00, page=1, limit=10
- **Response (JSON - 200 OK):**


```
{
 "status": "success",
 "message": "Lấy dữ liệu thành công",
 "pagination": {
  "current_page": 1,
  "total_pages": 5,
  "total_records": 50
 },
 "data": [
  {
   "id": 101,
   "sensor_id": 1,
   "name": "Nhiệt độ",
   "value": 28.0,
   "unit": "°C",
   "created_at": "2026-08-25T10:00:00Z"
  },
  {
   "id": 102,
   "sensor_id": 2,
   "name": "Độ ẩm",
```

```
      "value": 80.5,
      "unit": "%",
      "created_at": "2026-08-25T10:00:00Z"
    }
  ]
}
```


Màn hình sử dụng API và Use case:



Hình 3.1.4.3: *Giao diện sử dụng API, Use case*

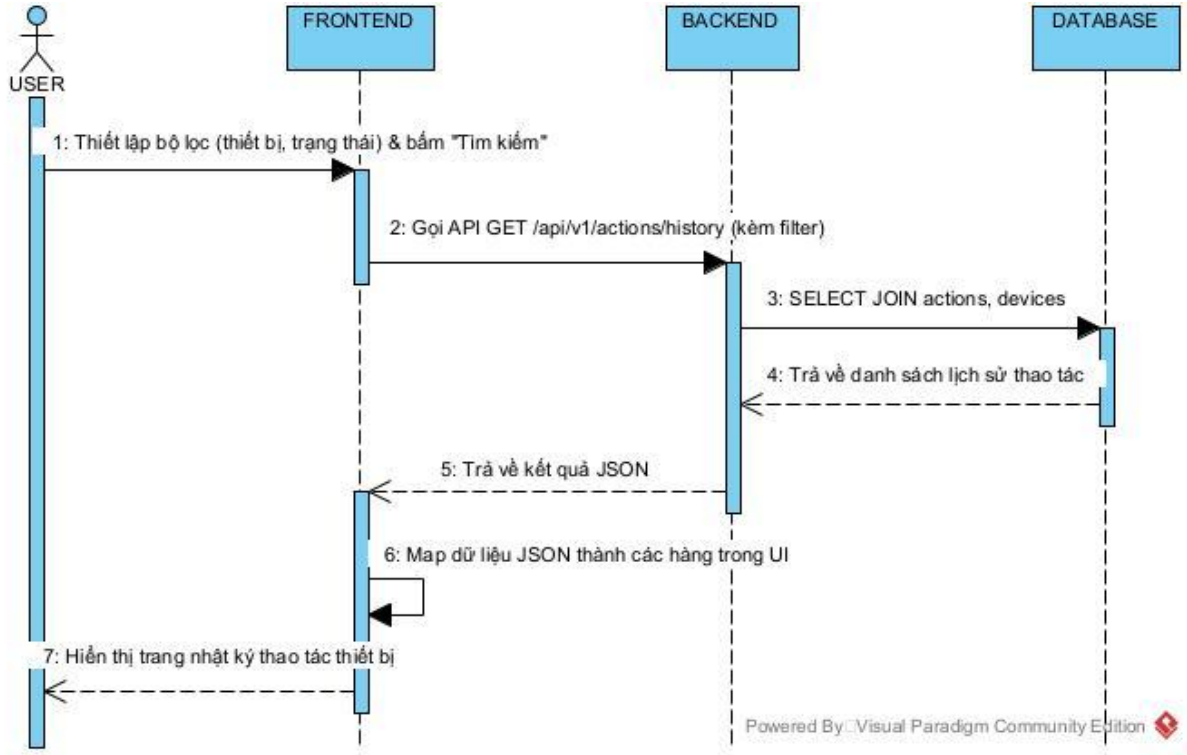
3.1.5. UC Tra cứu lịch sử thao tác thiết bị

3.1.5.1. Mô tả

Mã UC	UC – 05
Tên Use case	Tra cứu lịch sử thao tác thiết bị
Tác nhân	User
Mô tả	Tìm kiếm và xem lại toàn bộ nhật ký các lệnh điều khiển thiết bị trong quá khứ, hỗ trợ lọc theo thiết bị hoặc trạng thái (SUCCESS, FAILED)
Tiền điều kiện	Hệ thống Backend (BE) đang hoạt động ổn định. Bảng actions trong Cơ sở dữ liệu (DB) đã có sẵn các bản ghi lịch sử điều khiển từ các lần thao tác trước.

Hậu điều kiện	Người dùng xem được bảng nhật ký các lần thao tác thiết bị
Luồng sự kiện chính	1. Người dùng truy cập vào màn hình "Lịch sử hoạt động thiết bị". 2. Người dùng thiết lập bộ lọc tìm kiếm: chọn thiết bị (Tất cả / Đèn / Quạt), chọn trạng thái (SUCCESS / FAILED) 3. Frontend gửi request lấy danh sách nhật ký tới Backend. 4. Backend truy vấn kết hợp các bảng actions, devices và users, trả về danh sách lịch sử có cấu trúc kèm thông tin phân trang. 5. Người dùng quan sát bảng nhật ký chi tiết hiển thị trên màn hình gồm: Mã thao tác, Tên thiết bị, Hành động, Người thực hiện, Trạng thái và Thời gian ghi nhận.
Luồng ngoại lệ	1. Lỗi kết nối API: FE hiển thị thông báo lỗi "Không thể tải danh sách nhật ký thiết bị" và hiển thị nút "Thử lại". 2. Không có bản ghi phù hợp: Nếu không có bản ghi phù hợp nào, Backend trả về rỗng [] kèm total_records = 0. FE hiển thị bảng trống kèm thông báo "Không có lịch sử thao tác nào phù hợp với bộ lọc".

3.1.5.2. Sơ đồ tuần tự



Hình 3.1.5.2: *Sequence Digram Tra cứu lịch sử thao tác thiết bị*

Mô tả chi tiết luồng:

1. **Người dùng** truy cập vào menu "Lịch sử thiết bị" (Action History) trên thanh điều hướng. Thiết lập bộ lọc và nhấn “Tìm kiếm”
2. **Frontend** thu thập giá trị ghép thành query và gọi API GET /api/v1/actions/history gửi tới Backend
3. **Backend** bóc tách dữ liệu các tham số từ request và thực thi câu lệnh SQL vào bảng action trong cơ sở dữ liệu
4. **Cơ sở dữ liệu** xử lý truy vấn kết hợp nhiều bảng và trả lại danh sách các bản ghi lịch sử cùng tổng số dòng cho Backend.
5. **Backend** chuẩn hóa kết quả thành định dạng mảng JSON kèm metadata phân trang (current_page, total_pages, total_records) và phản hồi về cho Frontend.
6. **Frontend** tiếp nhận mảng JSON, thực thi hàm map dữ liệu để ánh xạ từng phần tử thành các hàng trong bảng nhật ký trên giao diện.
7. **Giao diện** hoàn thành việc render, hiển thị bảng danh sách lịch sử thao tác thiết bị chi tiết lên màn hình cho người dùng xem.

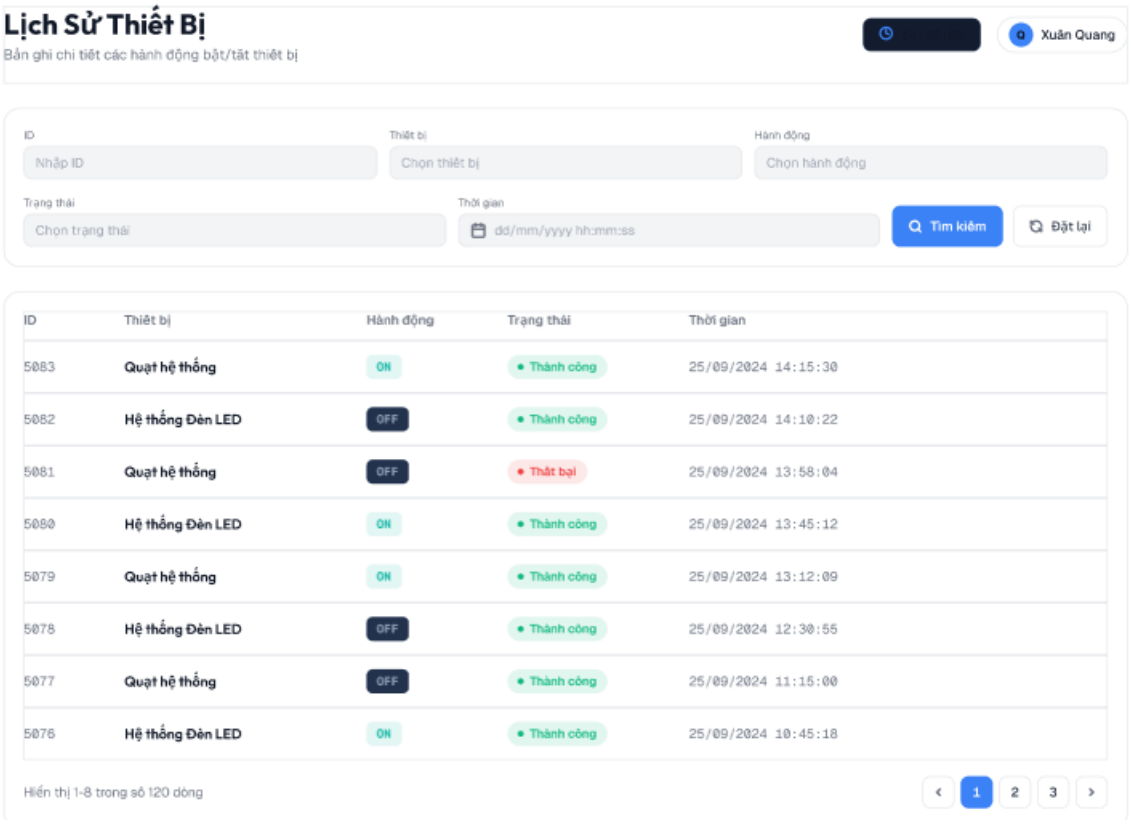
3.1.5.3. API Xem danh sách và tra cứu lịch sử thiết bị

- **Mô tả:** Lấy danh sách lịch sử bật/tắt thiết bị, hỗ trợ lọc theo thiết bị, trạng thái (SUCCESS/FAILED), mốc thời gian (time) và phân trang.

- **Method:** GET
- **Endpoint:** /api/v1/actions/history
- **Headers:** Authorization: Bearer <token>
- **Query Parameters:** page=1, limit=20, device_id=1, status=SUCCESS, time=2026-08-25
- **Response (JSON - 200 OK):**


```
{
 "status": "success",
 "pagination": {
  "current_page": 1,
  "total_pages": 2,
  "total_records": 15
 },
 "data": [
  {
   "id": 500,
   "device_id": 1,
   "device_name": "Đèn 1",
   "action": "TURN_ON",
   "user_name": "admin",
   "status": "SUCCESS",
   "created_at": "2026-08-25T09:30:00Z"
  },
  {
   "id": 499,
   "device_id": 2,
   "device_name": "Đèn 2 2",
   "action": "TURN_OFF",
   "user_name": "admin",
   "status": "SUCCESS",
   "created_at": "2026-08-25T08:15:00Z"
  }
 ]
}
```

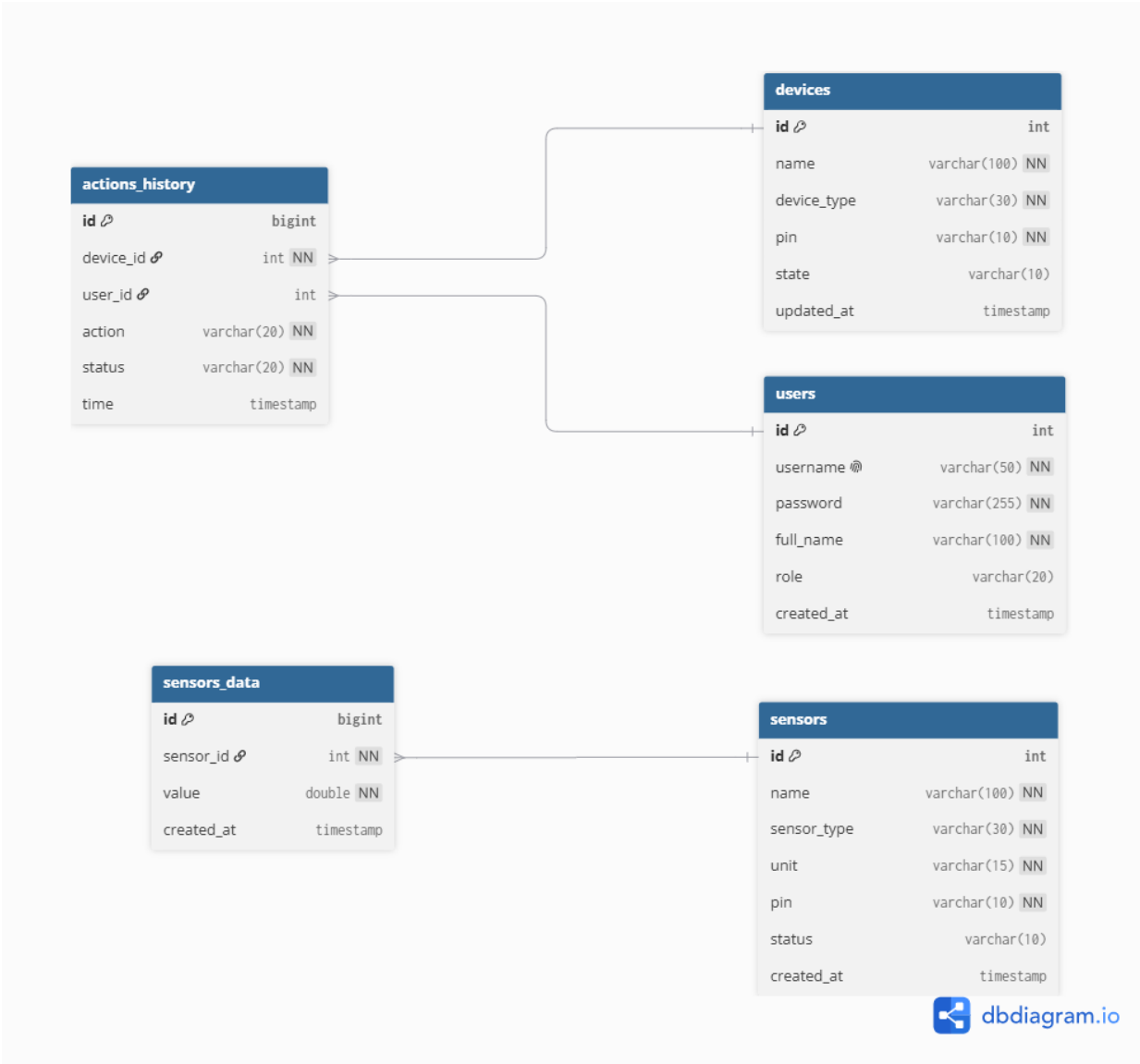
Màn hình sử dụng API và Use case:



Hình 3.1.5.3: *Giao diện sử dụng API, Use case*

3.2. Thiết kế cơ sở dữ liệu

Sơ đồ thực thể quan hệ



Hình 3.2: *Sơ đồ thực thể quan hệ*

1. Bảng User (Quản lý người dùng)

Tên cột	Kiểu dữ liệu	Khóa	Nullable	Ràng buộc	Ý nghĩa
Id	Int	PK	No	AUTO_INCREMENT	Định danh duy nhất của người dùng
Username	Varchar(50)		No	UNIQUE	Tên tài khoản đăng nhập
Password	Varchar(255)		No		Mật khẩu đã băm
Full_name	Varchar(100)		No		Họ tên hiển thị
Role	Varchar(20)		Yes	DEFAULT 'USER'	Quyền hạn (Admin/User)
Created_at	Timestamp		Yes	DEFAULT CURRENT_TIMESTAMP	Thời điểm tạo tài khoản

2. Bảng Devices (Danh sách thiết bị)

Tên cột	Kiểu dữ liệu	Khóa	Nullable	Ràng buộc	Ý nghĩa
Id	Int	PK	No	AUTO_INCREMENT	Mã định danh thiết bị
Name	Varchar(100)		No		Tên thiết bị
Device_type	Varchar(30)		No		Loại thiết bị: LIGHT, FAN, ...
Pin	Varchar(10)		No		Chân GPIO kích Relay
State	Varchar(10)		Yes	DEFAULT 'OFF'	Trạng thái hiện tại: ON/OFF
Updated_at	Timestamp		Yes	DEFAULT CURRENT_TIMESTAMP ON UPDATE CURRENT_TIMESTAMP	Thời điểm cập nhật trạng thái gần nhất

3. Bảng Sensors (Danh sách cảm biến)

Tên cột	Kiểu dữ liệu	Khóa	Nullable	Ràng buộc	Ý nghĩa
Id	Int	PK	No	AUTO_INCREMENT	Mã định danh cảm biến
Name	Varchar(100)		No		Tên cảm biến
Sensor_Type	Varchar(30)		No		Phân loại
Unit	Varchar(15)		No		Đơn vị đo: °C, %, lux

Pin	Varchar(10)		No		Chân GPIO đọc tín hiệu trên ESP32/8266
Status	Varchar(10)		Yes	DEFAULT 'ACTIVE'	Trạng thái cảm biến
Created_at	Timestamp		Yes	DEFAULT CURRENT_TIMESTAMP	Thời điểm cấu hình cảm biến

4. Bảng Sensors_Data (Lịch sử dữ liệu cảm biến)

Tên cột	Kiểu dữ liệu	Khóa	Nullable	Ràng buộc	Ý nghĩa
Id	Bigint	PK	No	AUTO_INCREMENT	Định danh bản ghi đo
Sensor_id	Int	FK	No	REFERENCES sensors(id) ON DELETE CASCADE	Cảm biến sinh ra dữ liệu
Value	Double		No		Giá trị đo được
Created_at	Timestamp		Yes	DEFAULT CURRENT_TIMESTAMP, INDEX	Thời điểm ghi nhận dữ liệu

5. Bảng Actions_History (Nhật ký lịch sử Bật/Tắt thiết bị)

Tên cột	Kiểu dữ liệu	Khóa	Nullable	Ràng buộc	Ý nghĩa
Id	Bigint	PK	No	AUTO_INCREMENT	Định danh bản ghi thao tác
Device_id	Int	FK	No	REFERENCES devices(id) ON DELETE CASCADE	Thiết bị được điều khiển
User_id	Int	FK	Yes	REFERENCES users(id) ON DELETE SET NULL	Người thực hiện (Null nếu do hệ thống tự kích hoạt)
Action	Varchar(20)		No		Lệnh thực hiện: TURN_ON, TURN_OFF
Status	Varchar(20)		No		Kết quả thực thi: PENDING, SUCCESS, FAILED

Time	Timestamp		Yes	DEFAULT CURRENT_TIMESTAMP, INDEX	Thời điểm phát sinh hành động

3.3. Tài liệu đặc tả API

3.3.1. Danh sách API tổng quan

Base URL: http://localhost:5000/api/v1

Content-Type: application/json

Cơ chế xác thực (Authentication): Hệ thống không bắt buộc Token xác thực để phục vụ quá trình chấm bài và kiểm thử trực tiếp trên Postman / Web nội bộ.

Bảng tổng hợp danh sách API thực tế của hệ thống Backend (IOT-Smart-Home):

STT	Mã API	Method	Endpoint	Mô tả ngắn gọn	Use Case
1	API-01	GET	/sensor/realtime	Lấy dữ liệu cảm biến thời gian thực (Nhiệt độ, Độ ẩm, Ánh sáng) kèm mốc thời gian	UC01
2	API-02	GET	/sensors/history	Lấy danh sách lịch sử dữ liệu cảm biến (lọc theo loại/tìm kiếm/sắp xếp/phân trang/biểu đồ)	UC02, UC04
3	API-03	GET	/devices/status	Lấy danh sách 2 thiết bị ngoại vi (Đèn 1, Đèn 2) và trạng thái ON/OFF hiện tại	UC03
4	API-04	POST	/devices/:id/control	Gửi lệnh Bật/Tắt thiết bị ngoại vi kèm phát bản tin MQTT tới ESP8266	UC03
5	API-05	GET	/actions/history	Lấy danh sách lịch sử thao tác bật/tắt thiết bị (phân trang, lọc trạng thái, thời gian)	UC05
6	API-06	GET	/user/profile	Lấy thông tin cá nhân sinh viên Đặng Xuân Quang và liên kết tài nguyên dự án	UC06
7	API-07	PUT	/user/profile	Cập nhật thông tin cá nhân và chỉnh sửa các liên kết tài nguyên nộp bài	UC06

3.3.2. Chi tiết đặc tả từng API

3.3.2.1. API-01: Lấy dữ liệu cảm biến thời gian thực (Realtime Sensor Data)

Thuộc tính	**Nội dung chi tiết**
API ID	API-01
Tên API	Lấy dữ liệu cảm biến thời gian thực
Method	GET
Endpoint	/sensor/realtime
Mô tả	Truy vấn giá trị đo mới nhất của Nhiệt độ (sensor_id=1), Độ ẩm (sensor_id=2) và Ánh sáng (sensor_id=3) từ bảng sensors_data trong CSDL MySQL, trả về phản hồi JSON tức thời cho client.
Xác thực (Auth)	Không yêu cầu
Use Case	UC01 (Giám sát cảm biến thời gian thực)
Tương tác CSDL	Đọc 1 dòng mới nhất cho từng sensor_id từ bảng sensors_data (SELECT ... ORDER BY created_at DESC, id DESC LIMIT 1)
Tương tác MQTT	Đọc dữ liệu tức thời được cập nhật liên tục qua dịch vụ mqttService

Request: Không có tham số.

Response Body mẫu (HTTP 200 OK):

```
{
  "status": "success",
  "data": {
    "temperature": 28.5,
    "humidity": 65.0,
    "light": 720,
    "timestamp": "2026-10-06T14:40:00.000Z"
  }
}
```

Mã trạng thái HTTP:

Code	**Ý nghĩa**
200	Lấy dữ liệu cảm biến thành công.
500	Lỗi máy chủ Backend hoặc mất kết nối CSDL MySQL.

3.3.2.2. API-02: Lấy danh sách lịch sử dữ liệu cảm biến

Thuộc tính	**Nội dung chi tiết**
API ID	API-02
Tên API	Lấy danh sách lịch sử dữ liệu cảm biến

Method	GET
Endpoint	/sensors/history (hoặc /sensor/history)
Mô tả	Truy vấn dữ liệu đo lưu trong bảng sensors_data kết hợp bảng sensors, hỗ trợ phân trang, lọc theo loại cảm biến (sensor_type), lọc giá trị (value), lọc mốc thời gian (time), và lấy tập dữ liệu vẽ biểu đồ khi tham số chart=true.
Xác thực (Auth)	Không yêu cầu
Use Case	UC02 (Giám sát biểu đồ cảm biến), UC04 (Tra cứu lịch sử cảm biến)
Tương tác CSDL	Truy vấn bảng sensors_data JOIN sensors với điều kiện lọc động và phân trang LIMIT/OFFSET
Tương tác MQTT	Không

Tham số truy vấn:

- **page (integer, tùy chọn):** Trang hiện tại (mặc định page=1).
- **limit (integer, tùy chọn):** Số bản ghi trên mỗi trang (mặc định limit=10).
- **sensor_type (string, tùy chọn):** Lọc theo loại cảm biến (TEMPERATURE, , LIGHT).
- **sensor_id (integer, tùy chọn):** Lọc theo ID cảm biến (1, 2, 3).
- **value (number, tùy chọn):** Lọc theo giá trị đo chính xác.
- **time (string, tùy chọn):** Lọc tìm kiếm theo ngày/giờ (định dạng YYYY-MM-DD hoặc DD/MM/YYYY).
- **chart (boolean, tùy chọn):** Trích xuất mảng dữ liệu phục vụ vẽ biểu đồ ApexCharts (mặc định false).

Response Body mẫu (HTTP 200 OK - Danh sách phân trang):


```
{
 "status": "success",
 "message": "Lấy dữ liệu thành công",
 "pagination": {
  "current_page": 1,
  "total_pages": 150,
  "total_records": 1500
 },
 "data": [
  {
   "id": 1500,
   "sensor_id": 1,
   "name": "Nhiệt độ",
   "sensor_type": "TEMPERATURE",
   "value": 28.5,
   "unit": "°C",
```

```
      "created_at": "2026-10-06 14:40:00"
    },
    {
      "id": 1499,
      "sensor_id": 2,
      "name": "Độ ẩm",
      "sensor_type": "HUMIDITY",
      "value": 65.0,
      "unit": "%",
      "created_at": "2026-10-06 14:40:00"
    },
    {
      "id": 1498,
      "sensor_id": 3,
      "name": "Ánh sáng",
      "sensor_type": "LIGHT",
      "value": 720.0,
      "unit": "lux",
      "created_at": "2026-10-06 14:40:00"
    }
  ]
}
```


Mã trạng thái HTTP:

Code	**Ý nghĩa**
200	Lấy danh sách thành công.
500	Lỗi máy chủ Backend hoặc CSDL MySQL.

3.3.2.3. API-03: Lấy danh sách thiết bị ngoại vi và trạng thái

Thuộc tính	**Nội dung chi tiết**
API ID	API-03
Tên API	Lấy danh sách thiết bị và trạng thái
Method	GET
Endpoint	/devices/status
Mô tả	Đọc danh sách 2 thiết bị ngoại vi (Đèn 1, Đèn 2) cùng thông tin chân GPIO, loại thiết bị và trạng thái ON/OFF từ bảng devices.
Xác thực (Auth)	Không yêu cầu
Use Case	UC03 (Điều khiển thiết bị)
Tương tác CSDL	Bảng devices (SELECT id, name, device_type, pin, state, updated_at FROM devices ORDER BY id ASC)

Tương tác MQTT	Không

Request: Không có tham số.

Response Body mẫu (HTTP 200 OK):


```
{
 "status": "success",
 "data": [
  {
   "id": 1,
   "name": "Đèn 1",
   "device_type": "LIGHT",
   "pin": "D1",
   "state": "ON",
   "updated_at": "2026-10-06T14:40:00.000Z"
  },
  {
   "id": 2,
   "name": "Đèn 2",
   "device_type": "LIGHT",
   "pin": "D2",
   "state": "OFF",
   "updated_at": "2026-10-06T14:35:00.000Z"
  }
 ]
}
```


Mã trạng thái HTTP:

Code	**Ý nghĩa**
200	Lấy trạng thái thiết bị thành công.
500	Lỗi máy chủ Backend.

3.3.2.4. API-04: Điều khiển thiết bị ngoại vi

Thuộc tính	**Nội dung chi tiết**
API ID	API-04
Tên API	Điều khiển bật/tắt thiết bị ngoại vi
Method	POST
Endpoint	/devices/:id/control (Ví dụ: /devices/1/control)
Mô tả	Tiếp nhận lệnh điều khiển, lưu nhật ký thao tác PENDING vào bảng actions_history, phát bản tin MQTT (ví dụ 'LED1_ON', 'LED1_OFF',

	'LED2_ON', 'LED2_OFF') xuống ESP8266 và cập nhật trạng thái thiết bị trong CSDL.
Xác thực (Auth)	Không yêu cầu
Use Case	UC03 (Điều khiển thiết bị)
Tương tác CSDL	Bảng devices (kiểm tra tồn tại và cập nhật state), Bảng actions_history (thêm bản ghi mới)
Tương tác MQTT	Publish chuỗi lệnh điều khiển qua MQTT Broker (Mosquitto)

Tham số đường dẫn:

- **id (integer, bắt buộc):** ID thiết bị (1: Đèn 1, 2: Đèn 2).

Request Body mẫu (JSON):

```
{
 "action": "TURN_ON"
}
```

Response Body mẫu (HTTP 200 OK):

```
{
 "status": "success",
 "message": "Điều khiển thiết bị thành công",
 "data": {
  "action_id": 120,
  "device_id": 1,
  "device_name": "Đèn 1",
  "state": "ON",
  "updated_at": "2026-10-06T14:40:00.000Z"
 }
}
```

Mã trạng thái HTTP:

Code	**Ý nghĩa**
200	Điều khiển thiết bị thành công.
400	Thiếu trường dữ liệu hoặc hành động (action) không hợp lệ.
404	Không tìm thấy thiết bị có ID tương ứng.
500	Lỗi máy chủ Backend hoặc mất kết nối MQTT Broker.

3.3.2.5. API-05: Lấy danh sách lịch sử thao tác thiết bị

Thuộc tính	**Nội dung chi tiết**
API ID	API-05
Tên API	Lấy lịch sử thao tác điều khiển thiết bị

Method	GET
Endpoint	/actions/history
Mô tả	Truy vấn toàn bộ lịch sử bật/tắt thiết bị từ bảng actions_history kết hợp bảng devices và users, hỗ trợ phân trang và lọc theo điều kiện.
Xác thực (Auth)	Không yêu cầu
Use Case	UC05 (Tra cứu lịch sử thao tác thiết bị)
Tương tác CSDL	Bảng actions_history JOIN devices LEFT JOIN users (SELECT ... ORDER BY ah.time DESC, ah.id DESC)
Tương tác MQTT	Không

Tham số truy vấn:

- **page (integer, tùy chọn):** Trang hiện tại (mặc định page=1).
- **limit (integer, tùy chọn):** Số bản ghi trên trang (mặc định limit=10).
- **device_id (integer, tùy chọn):** Lọc theo ID thiết bị (1, 2).
- **status (string, tùy chọn):** Lọc theo trạng thái thực thi (SUCCESS, FAILED, PENDING).
- **action (string, tùy chọn):** Lọc theo hành động (TURN_ON, TURN_OFF).
- **time (string, tùy chọn):** Lọc theo mốc thời gian thực hiện.

Response Body mẫu (HTTP 200 OK):


```
{
 "status": "success",
 "pagination": {
  "current_page": 1,
  "total_pages": 12,
  "total_records": 120
 },
 "data": [
  {
   "id": 120,
   "device_id": 1,
   "device_name": "Đèn 1",
   "action": "TURN_ON",
   "user_name": "Đặng Xuân Quang",
   "status": "SUCCESS",
   "created_at": "2026-10-06 14:38:15"
  },
  {
   "id": 119,
   "device_id": 2,
   "device_name": "Đèn 2",
```

```
      "action": "TURN_OFF",
      "user_name": "Đặng Xuân Quang",
      "status": "SUCCESS",
      "created_at": "2026-10-06 14:35:10"
    }
  ]
}
```


Mã trạng thái HTTP:

Code	Ý nghĩa
200	Lấy danh sách nhật ký thao tác thành công.
500	Lỗi máy chủ Backend hoặc CSDL MySQL.

3.3.2.6. API-06: Lấy thông tin hồ sơ người dùng

Thuộc tính	Nội dung chi tiết
API ID	API-06
Tên API	Lấy thông tin hồ sơ người dùng và liên kết nộp bài
Method	GET
Endpoint	/user/profile
Mô tả	Đọc thông tin cá nhân của sinh viên Đặng Xuân Quang cùng 4 đường liên kết tài nguyên nộp bài (GitHub, Figma, Postman, Báo cáo).
Xác thực (Auth)	Không yêu cầu
Use Case	UC06 (Quản lý Hồ sơ cá nhân và Liên kết tài nguyên)
Tương tác CSDL	Không
Tương tác MQTT	Không

Request: Không có tham số.

Response Body mẫu (HTTP 200 OK):


```
{
  "status": "success",
  "data": {
    "id": 1,
    "full_name": "Đặng Xuân Quang",
    "student_id": "B23DCCN686",
    "class_name": "D23CNPM06",
    "major": "Công nghệ phần mềm",
    "email": "quangdx.b23cn686@stu.ptit.edu.vn",
    "avatar_url": "https://ui-
```

avatars.com/api/?name=Dang+Xuan+Quang&background=3b82f6&color=fff&size=128",
"links": {
"github": "https://github.com/XuanQuang1301/IOT-Smart-Home",
"figma": "https://www.figma.com/design/bp3cfY9WkXlfF5lN4PreqN/Untitled?node-id=0-1&t=FEGKawTyiA9gA240-1",
"postman": "https://postman.com",
"report": "https://drive.google.com/file/d/13nv0bGtUUwD4Xyn19TzxqBvjHtVGuRMv/view?usp=sharing"
}
}
}

Mã trạng thái HTTP:

Code	Ý nghĩa
200	Lấy thông tin hồ sơ thành công.
500	Lỗi máy chủ Backend.

3.3.2.7. API-07: Cập nhật thông tin hồ sơ người dùng (Update User Profile)

Thuộc tính	Nội dung chi tiết
API ID	API-07
Tên API	Cập nhật thông tin hồ sơ người dùng và liên kết nộp bài
Method	PUT
Endpoint	/user/profile
Mô tả	Cập nhật các trường thông tin cá nhân sinh viên và liên kết tài liệu nộp bài.
Xác thực (Auth)	Không yêu cầu
Use Case	UC06 (Quản lý Hồ sơ cá nhân và Liên kết tài nguyên)
Tương tác CSDL	Không
Tương tác MQTT	Không

Request Body mẫu (JSON):

```
{
 "full_name": "Đặng Xuân Quang",
 "student_id": "B23DCCN686",
 "class_name": "D23CNPM06",
 "major": "Công nghệ phần mềm"
}
```

Response Body mẫu (HTTP 200 OK):

```
{
 "status": "success",
 "message": "Cập nhật thông tin hồ sơ thành công",
 "data": {
  "full_name": "Đặng Xuân Quang",
  "student_id": "B23DCCN686",
  "class_name": "D23CNPM06",
  "major": "Công nghệ phần mềm",
  "id": 1
 }
}
```

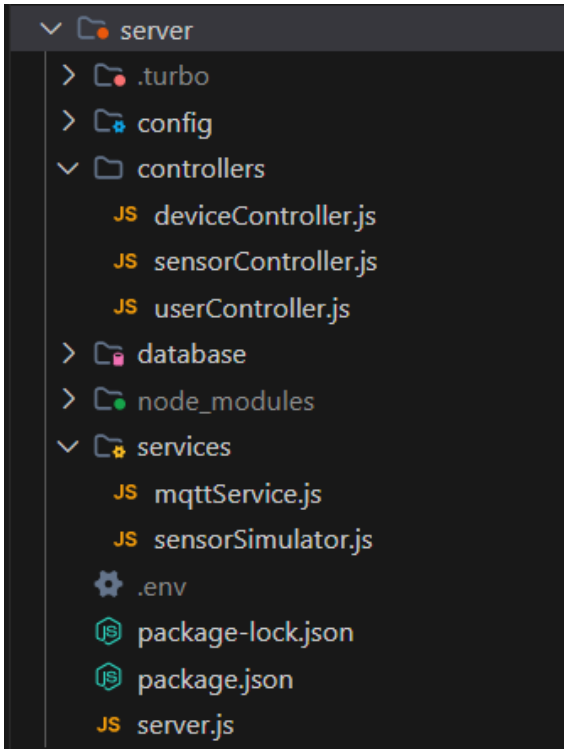

Mã trạng thái HTTP (HTTP Status):

Code	Ý nghĩa
200	Cập nhật hồ sơ thành công.
400	Dữ liệu gửi lên không hợp lệ.
500	Lỗi máy chủ Backend.

CHƯƠNG 4: KẾT QUẢ VÀ ĐÁNH GIÁ THỰC NGHIỆM

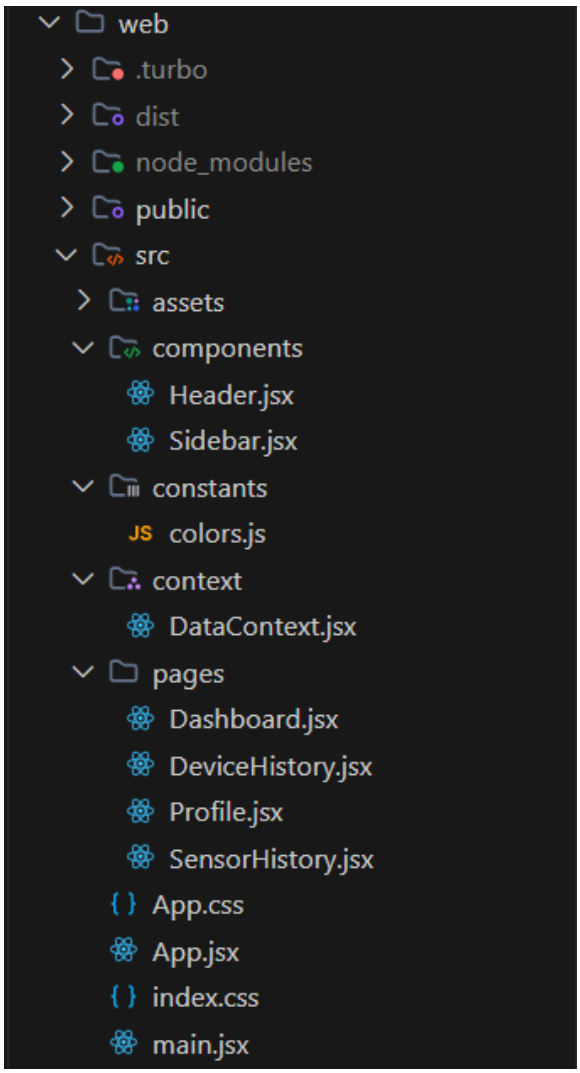
4.1. Cấu trúc thư mục dự án

Cấu trúc thư mục Backend:



***Hình 4.1.1**: Cấu trúc thư mục Backend*

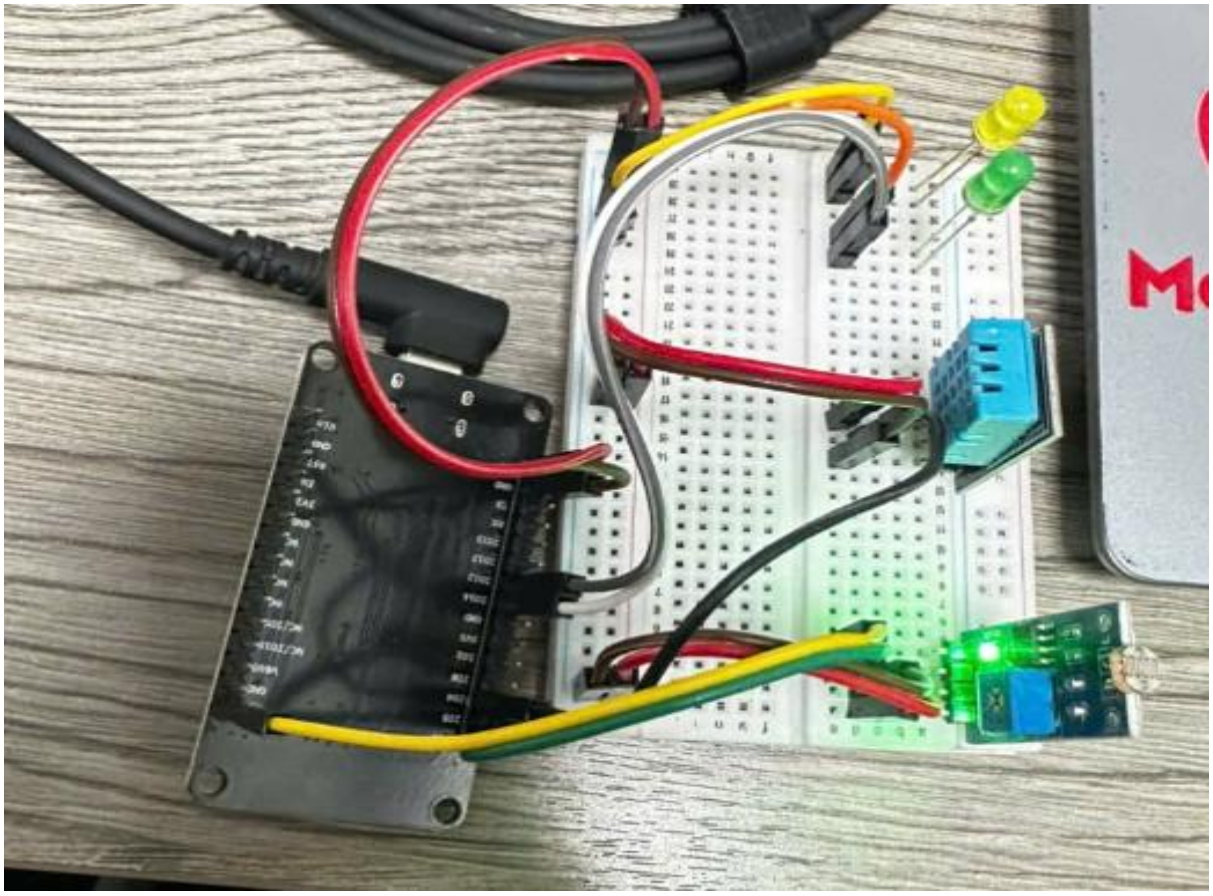
Cấu trúc thư mục Frontend:



***Hình 4.1.2**: Cấu trúc thư mục Frontend*

4.2. Kết quả triển khai phần cứng và MQTT

4.2.1. Triển khai phần cứng vi điều khiển



Hình 4.2.1: *Hình ảnh mạch*

- Kit vi điều khiển: ESP8266 NodeMCU V3 (dòng chip ESP-12E, tốc độ xử lý 80/160 MHz, tích hợp module Wi-Fi chuẩn 802.11 b/g/n).

- **Cảm biến nhiệt độ** - độ ẩm DHT11: Kết nối chân tín hiệu Data với chân GPIO 4 (D2) của ESP8266. Chu kỳ lấy mẫu cảm biến được ấn định ổn định là 2.0 giây/lần.

- **Cảm biến cường độ ánh sáng quang trở** (LDR): Đọc giá trị điện áp tương tự thông qua chân ADC A0. Dải đo ADC 10-bit từ 0 đến 1023 được tính toán chuyển đổi thành giá trị cường độ ánh sáng tỉ lệ nghịch.

4.2.2. Giao thức truyền thông MQTT và quản lý phiên

MQTT Broker: Sử dụng Mosquitto MQTT Broker (port 6868), kết nối ổn định giữa ESP8266 và Backend Node.js.

Topic dữ liệu cảm biến (Telemetry Uplink): iot/sensors/data

```
{
 "temperature": 28.5,
 "humidity": 65.0,
```

```
  "light": 720
}
```

Định kỳ **2000ms**, ESP8266 gửi dữ liệu đo môi trường dạng JSON lên Broker.

Topic điều khiển (Command Downlink): iot/devices/control

LED1_ON / LED1_OFF / LED2_ON / LED2_OFF

Backend phát lệnh bật/tắt chân GPIO tương ứng xuống phần cứng ESP8266.

4.3. Kết quả triển khai Backend và cơ sở dữ liệu

4.3.1. Cấu hình cơ sở dữ liệu MySQL

Sử dụng CSDL quan hệ MySQL thông qua thư viện mysql2 với cơ chế Connection Pool tối ưu hiệu năng truy vấn:

- **users:** Lưu thông tin sinh viên thực hiện.
- **sensors:** Danh mục cảm biến trong hệ thống (Nhiệt độ, Độ ẩm, Ánh sáng).
- **devices:** Danh mục 2 thiết bị ngoại vi (Đèn 1, Đèn 2) và trạng thái ON/OFF hiện tại.
- **sensors_data:** Lưu trữ toàn bộ lịch sử dữ liệu đo cảm biến theo thời gian thực.
- **actions_history:** Lưu vết lịch sử thao tác bật/tắt thiết bị của người dùng.

4.3.2. Tài liệu API và kiểm thử Endpoint

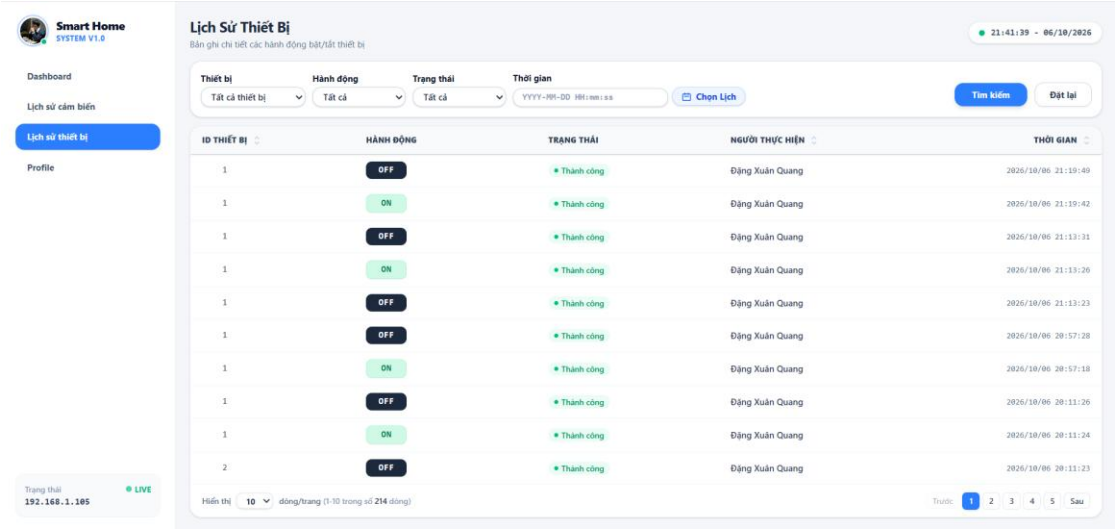
Hệ thống 7 RESTful API chính đã được kiểm thử toàn diện trên Postman và chạy thực tế trơn tru:

- **200 OK:** Trả về dữ liệu JSON { status: 'success', data: ... }.
- **400 Bad Request:** Bắt lỗi tham số hoặc action không hợp lệ.
- **404 Not Found:** Bắt lỗi khi gọi ID thiết bị không tồn tại.
- **500 Internal Server Error:** Xử lý ngoại lệ kết nối CSDL hoặc Broker.

4.4. Kết quả xây dựng giao diện Frontend

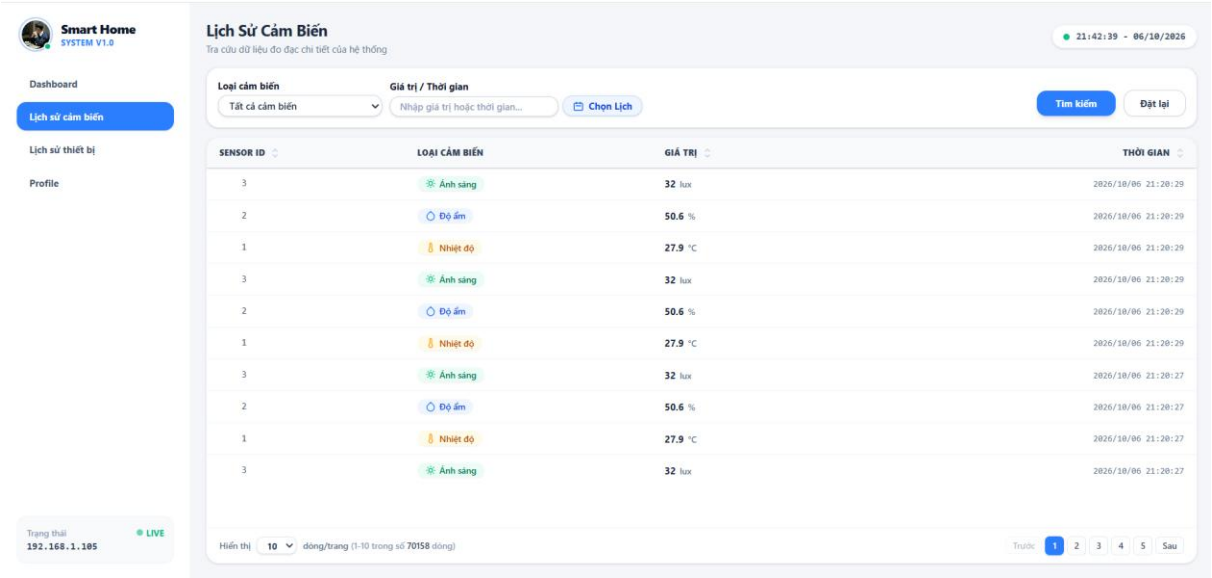
Giao diện Web được phát triển bằng React, TailwindCSS và Vite với 4 màn hình chính:

1. **Màn hình Tổng quan (Dashboard):** Hiển thị 3 thẻ số liệu cảm biến (Nhiệt độ, Độ ẩm, Ánh sáng), biểu đồ ApexCharts đa trục thời gian thực (polling 2s/lần) và bộ nút gạt công tắc điều khiển 2 thiết bị ngoại vi (Đèn 1, Đèn 2).



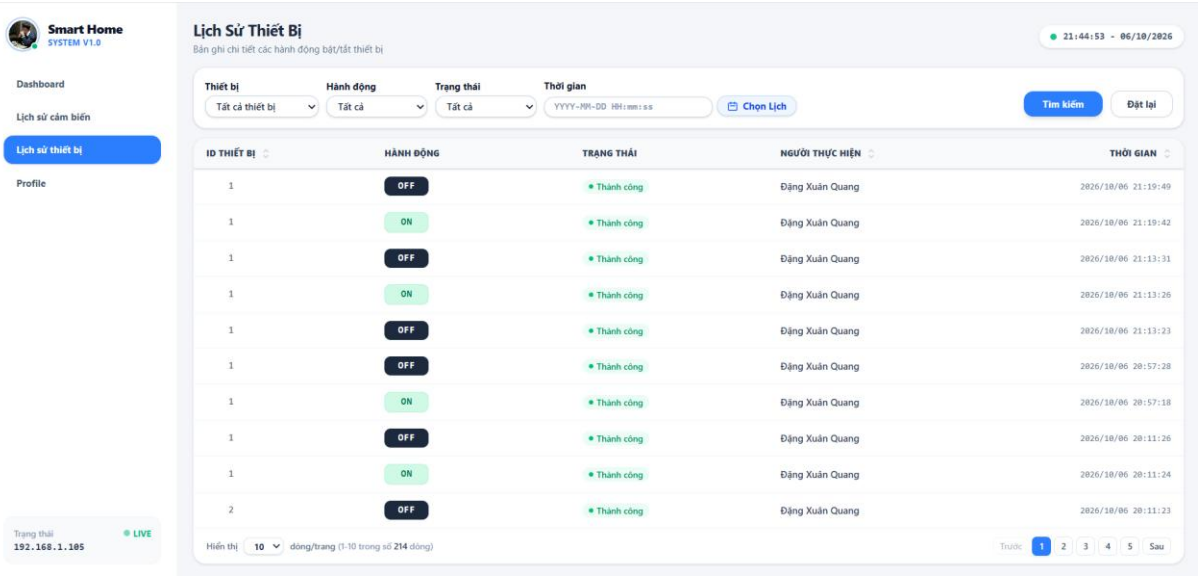
Hình 4.4.1: *Giao diện DashBoard*

2. **Màn hình Dữ liệu cảm biến (SensorHistory):** Bảng tra cứu lịch sử đo cảm biến với bộ lọc loại cảm biến, giá trị, tìm kiếm ngày giờ và phân trang linh hoạt.



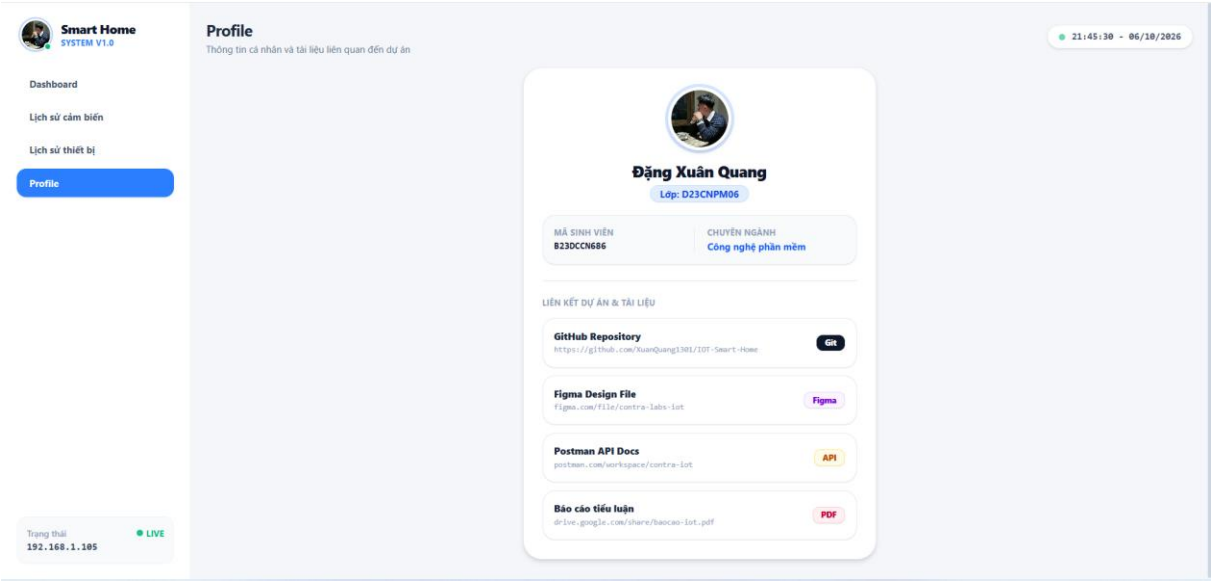
Hình 4.4.2: *Giao diện Lịch sử cảm biến*

3. **Màn hình Lịch sử thiết bị (DeviceHistory):** Bảng nhật ký lịch sử bật/tắt thiết bị kèm trạng thái SUCCESS / FAILED / PENDING và tìm kiếm theo mốc thời gian.



Hình 4.4.3: *Giao diện Lịch sử thiết bị*

4. **Màn hình Hồ sơ cá nhân (Profile):** Hiển thị thông tin sinh viên thực hiện (Đặng Xuân Quang - B23DCCN686 - D23CNPM06) và các liên kết tài nguyên dự án (GitHub, Figma, Postman, Báo cáo).



Hình 4.4.4: *Giao diện Profile*

4.5. Đánh giá thử nghiệm và kiểm thử hệ thống

Bảng đối chiếu kịch bản kiểm thử hệ thống:

STT	Kịch bản kiểm thử	Hành vi thiết kế trong hệ thống	Kết quả đối chiếu thực nghiệm	Trạng thái
1	Thu thập dữ liệu cảm biến định kỳ	ESP8266 đo DHT11 & LDR mỗi 2 giây và publish JSON qua MQTT.	Backend nhận tin, lưu vào bảng sensors_data và cập nhật RAM.	ĐẠT
2	Giám sát thời gian thực trên Dashboard	Frontend polling API GET /api/v1/sensor/realtime mỗi 2s.	Các thẻ số nhảy liên tục; biểu đồ ApexCharts tự động vẽ thêm điểm mới đúng chu kỳ 2s.	ĐẠT
3	Điều khiển 2 thiết bị (Đèn 1, Đèn 2) từ xa	Người dùng gạt nút công tắc; Frontend gửi POST /api/v1/devices/:id/control.	Backend ghi nhận actions_history (SUCCESS), bắn lệnh MQTT (LED1_ON/OFF, LED2_ON/OFF), LED đổi trạng thái tức thì.	ĐẠT
4	Tra cứu lịch sử cảm biến & Phân trang	Người dùng lọc dữ liệu cảm biến hoặc chuyển trang trên SensorHistory.	API xử lý truy vấn SQL phân trang (< 50ms), trả về kết quả chính xác theo bộ lọc.	ĐẠT
5	Tra cứu lịch sử thao tác thiết bị	Người dùng tra cứu nhật ký bật/tắt thiết bị trên DeviceHistory.	Hiển thị đầy đủ thiết bị (Đèn 1, Đèn 2), hành động TURN_ON/OFF, thời gian và trạng thái thực hiện.	ĐẠT

4.6. Hướng dẫn chạy và triển khai hệ thống

4.6.1. Yêu cầu môi trường

- **Node.js:** Phiên bản >= 18.x
- **MySQL:** Phiên bản >= 8.0
- **Mosquitto MQTT Broker:** Broker chạy tại localhost:6868

4.6.2. Các bước triển khai chi tiết

1. Khởi tạo Cơ sở dữ liệu: Chạy kịch bản SQL trong thư mục server/database/schema.sql để tạo database và các bảng ban đầu.

2. Khởi động Backend Server: Di chuyển vào thư mục server/, cài đặt thư viện (npm install) và chạy node server.js. Server sẽ khởi chạy tại: http://localhost:5000.

3. Khởi động Frontend Client: Di chuyển vào thư mục web/, cài đặt thư viện (npm install) và chạy npm run dev. Trình duyệt mở tại: http://localhost:5173.

CHƯƠNG 5: KẾT LUẬN

5.1. Kết luận và Kết quả đạt được

Đề tài ***'Hệ thống IoT giám sát môi trường và điều khiển thiết bị'*** do sinh viên Đặng Xuân Quang (MSV: B23DCCN686, Lớp: D23CNPM06) hoàn thành đã đáp ứng trọn vẹn toàn bộ các mục tiêu đề ra:

- Xây dựng phần cứng **ESP8266** kết nối cảm biến DHT11 và quang trở LDR, thu thập dữ liệu nhiệt độ, độ ẩm, ánh sáng chính xác mỗi 2 giây.
- Triển khai giao thức truyền thông **MQTT** tốc độ cao, điều khiển bật/tắt 2 thiết bị ngoại vi (Đèn 1, Đèn 2) với độ trễ thấp.
- Xây dựng máy chủ phụ trợ Node.js/Express cung cấp 7 RESTful API chuẩn mực, truy vấn CSDL MySQL tối ưu hiệu năng.
- Phát triển giao diện Web React & Vite hiện đại, trực quan với các chức năng Dashboard thời gian thực, tra cứu dữ liệu cảm biến, lịch sử thiết bị và thông tin cá nhân.
- Mã nguồn dự án được lưu trữ và quản lý công khai tại GitHub Repository: https://github.com/XuanQuang1301/IOT-Smart-Home.

5.2. Bài học và Kinh nghiệm rút ra

- **Tầm quan trọng của thiết kế API:** Thống nhất chuẩn định dạng dữ liệu API và Schema CSDL giữa Backend và Frontend giúp tối ưu thời gian phát triển và hạn chế lỗi ghép nối.
- **Tư duy kiến trúc hệ thống:** Tách biệt rõ ràng tầng Server và tầng Web giúp mã nguồn gọn gàng, dễ nâng cấp và bảo trì.
- **Nâng cao kỹ năng chuyên môn:** Rèn luyện kỹ năng lập trình React, Node.js, Express, MySQL và làm việc với các công cụ phát triển hiện đại.